

🚀 MasterMock Backend – Complete API Documentation

****Version:**** 1.0

****Base URL:**** `http://localhost:3000/api/v1`

****Last Updated:**** June 13, 2026

📄 Table of Contents

1. [General Information] ([#1-general-information](#))
2. [Authentication & Authorization] ([#2-authentication--authorization](#))
3. [Standard Response Format] ([#3-standard-response-format](#))
4. [Standard Error Format] ([#4-standard-error-format](#))
5. [Rate Limiting] ([#5-rate-limiting](#))
6. [Data Models (Schemas)] ([#6-data-models-schemas](#))
7. [API Endpoints] ([#7-api-endpoints](#))
 - [Health Check] ([#71-health-check](#))
 - [Auth Routes] ([#72-auth-routes](#))
 - [User Routes] ([#73-user-routes](#))
 - [Category Routes] ([#74-category-routes](#))
 - [Course Routes] ([#75-course-routes](#))
 - [Resource Routes] ([#76-resource-routes](#))
 - [Mock Test Routes] ([#77-mock-test-routes](#))
 - [Test Attempt Routes] ([#78-test-attempt-routes](#))
 - [Payment Routes] ([#79-payment-routes](#))
 - [Notification Routes] ([#710-notification-routes](#))
 - [Dashboard Routes] ([#711-dashboard-routes](#))
 - [Leaderboard Routes] ([#712-leaderboard-routes](#))
 - [Inquiry Routes] ([#713-inquiry-routes](#))
8. [Enum Values Reference] ([#8-enum-values-reference](#))
9. [Important Notes for Frontend] ([#9-important-notes-for-frontend](#))

1. General Information

Property	Value
-----	-----
Framework	Express.js (Node.js)

Database	MongoDB (Mongoose ODM)	
Content-Type	`application/json` (all requests/responses)	
File Uploads	`multipart/form-data` (resource upload only)	
Credentials	Cookies are used, set `withCredentials: true`	
Body Size Limit	16 KB (JSON/URL-encoded), 50 MB (file uploads)	
Payment Gateway	Razorpay	

2. Authentication & Authorization

How Authentication Works

The API uses ****JWT (JSON Web Token)**** for authentication with a ****dual-token system****:

- ****Access Token**** — Short-lived (default: 1 day). Used for API requests.
- ****Refresh Token**** — Long-lived (default: 10 days). Used to get new access tokens.

Sending the Access Token

The token can be sent in ****one of two ways**** (backend checks both):

...

Option 1: Authorization Header (RECOMMENDED)

Authorization: Bearer <accessToken>

Option 2: Cookies (set automatically on login)

Cookie: accessToken=<accessToken>; refreshToken=<refreshToken>

...

User Roles

Role	Description	
-----	-----	
`STUDENT`	Default role. Can view courses, take tests, etc.	
`ADMIN`	Full access. Can manage courses, tests, users.	
`INSTRUCTOR`	Reserved for future use.	

Access Markers Used in This Document

Marker	Meaning
-----	-----
🟢 **Public**	No authentication required
🔒 **Auth Required**	Must send a valid access token
🔴 **Admin Only**	Must be logged in with <code>`role: "ADMIN"`</code>

3. Standard Response Format

****All successful responses**** follow this JSON structure:

```
```json
{
 "statusCode": 200,
 "data": { ... },
 "message": "Success message here",
 "success": true
}
```
```

| Field | Type | Description |
|---------------------------|---------|---|
| ----- | ----- | ----- |
| <code>`statusCode`</code> | Number | HTTP status code (200, 201, etc.) |
| <code>`data`</code> | Any | The response payload (object, array, etc.) |
| <code>`message`</code> | String | Human-readable success message |
| <code>`success`</code> | Boolean | Always <code>`true`</code> for status < 400 |

4. Standard Error Format

****All error responses**** follow this JSON structure:

```
```json
{
 "success": false,
 "message": "Error message here",
}
```

```

 "errors": [],
 "stack": "... " // Only in development environment
 }
}

```

Field	Type	Description
-----	-----	
`success`	Boolean	Always `false`
`message`	String	Human-readable error message
`errors`	Array	Validation errors array (from Zod validation)
`stack`	String	Stack trace (only present in development mode)

### ### Common HTTP Error Codes

Code	Meaning
-----	-----
`400`	Bad Request / Validation Error
`401`	Unauthorized (no/invalid token)
`403`	Forbidden (wrong role / suspended / account locked)
`404`	Not Found
`409`	Conflict (duplicate entry)
`429`	Too Many Requests (rate limited)
`500`	Internal Server Error

---

## ## 5. Rate Limiting

Limiter	Scope	Limit
-----	-----	
-----		
<b>**Global Limiter**</b>	All `/api/*` routes	100 requests per 15 minutes per IP

```
| **Auth Limiter** | Login route only | 10 requests per 1 hour per
IP |
```

When rate limited, the API returns `429 Too Many Requests`.

---

## ## 6. Data Models (Schemas)

### ### 6.1 User

```
```json
{
  "_id": "ObjectId",
  "full_name": "String (required, 2-50 chars)",
  "email": "String (required, unique, lowercase)",
  "phone_number": "String (optional, unique)",
  "profile_picture": "String (URL, optional)",
  "status": "String - 'active' | 'suspended' | 'unverified' (default:
'unverified')",
  "role": "String - 'STUDENT' | 'ADMIN' | 'INSTRUCTOR' (default:
'STUDENT')",
  "createdAt": "ISO Date",
  "updatedAt": "ISO Date"
}
```
```

> ⚠️ `password\_hash` and `refresh\_token` are **\*\*never\*\*** returned in any API response.

### ### 6.2 Category

```
```json
{
  "_id": "ObjectId",
  "name": "String (required, unique, 2-100 chars)",
  "description": "String (optional)",
  "createdAt": "ISO Date",
  "updatedAt": "ISO Date"
}
```

6.3 Course

```
```json
{
 "_id": "ObjectId",
 "title": "String (required, min 3 chars)",
 "description": "String (required, min 10 chars)",
 "price": "Number (required, >= 0, default: 0)",
 "access_type": "String - 'free' | 'paid' (default: 'paid')",
 "category": "ObjectId (ref: Category) - populated as { _id, name }",
 "created_by": "ObjectId (ref: User)",
 "is_active": "Boolean (default: true)",
 "isDeleted": "Boolean (default: false)",
 "deletedAt": "Date | null",
 "createdAt": "ISO Date",
 "updatedAt": "ISO Date"
}
```
```

6.4 Enrollment

```
```json
{
 "_id": "ObjectId",
 "user": "ObjectId (ref: User)",
 "course": "ObjectId (ref: Course)",
 "status": "String - 'ACTIVE' | 'EXPIRED' | 'REVOKED' (default:
'ACTIVE')",
 "enrolled_at": "ISO Date",
 "access_expires_at": "Date | null",
 "createdAt": "ISO Date",
 "updatedAt": "ISO Date"
}
```
```

> A user can only be enrolled in a specific course once (unique index: `user + course`).

6.5 MockTest

```
```json
{
 "_id": "ObjectId",
 "title": "String (required, min 3 chars)",
 "description": "String (optional)",
 "course": "ObjectId (ref: Course, optional)",
 "category": "ObjectId (ref: Category, optional)",
 "difficulty": "String - 'easy' | 'medium' | 'hard' (default: 'medium')",
 "total_questions": "Number (required, >= 1)",
 "passing_marks": "Number (required, >= 1)",
 "negative_marking": "Boolean (default: false)",
 "negative_marks_per_wrong": "Number (default: 0)",
 "total_marks": "Number (required, >= 1)",
 "duration_minutes": "Number (required, >= 1)",
 "created_by": "ObjectId (ref: User)",
 "is_active": "Boolean (default: true)",
 "isDeleted": "Boolean (default: false)",
 "deletedAt": "Date | null",
 "questions": [
 {
 "_id": "ObjectId",
 "text": "String (required)",
 "marks": "Number (default: 1)",
 "explanation": "String (optional)",
 "options": [
 {
 "_id": "ObjectId",
 "text": "String (required)",
 "is_correct": "Boolean (required)"
 }
]
 }
],
 "createdAt": "ISO Date",
 "updatedAt": "ISO Date"
}
```
```

>  ****Important:**** For STUDENT users, the field `options.is\_correct` is ****stripped from the response**** (hidden). Only ADMIN users can see correct answers.

6.6 TestAttempt

```
```json
{
 "_id": "ObjectId",
 "user": "ObjectId (ref: User)",
 "mock_test": "ObjectId (ref: MockTest)",
 "started_at": "ISO Date",
 "completed_at": "ISO Date | null",
 "status": "String - 'IN_PROGRESS' | 'COMPLETED' | 'ABANDONED' (default: 'IN_PROGRESS')",
 "answers": [
 {
 "question_id": "ObjectId",
 "question_text": "String",
 "selected_option_id": "ObjectId | null",
 "selected_option_text": "String | null",
 "is_correct": "Boolean (default: false)",
 "is_marked_for_review": "Boolean (default: false)",
 "answered_at": "ISO Date"
 }
],
 "score": "Number (default: 0)",
 "percentage": "Number (default: 0)",
 "rank": "Number | null",
 "feedback": "String | null",
 "createdAt": "ISO Date",
 "updatedAt": "ISO Date"
}
```
```

6.7 Payment

```
```json
{
 "_id": "ObjectId",
```



```

 "user": "ObjectId (ref: User)",
 "amount": "Number (required)",
 "currency": "String (default: 'INR')",
 "razorpay_order_id": "String (required, unique)",
 "razorpay_payment_id": "String | null",
 "razorpay_signature": "String | null",
 "status": "String - 'PENDING' | 'SUCCESS' | 'FAILED' | 'REFUNDED'
(default: 'PENDING')",
 "createdAt": "ISO Date",
 "updatedAt": "ISO Date"
}
```

```

6.8 Purchase

```

```json
{
 "_id": "ObjectId",
 "user": "ObjectId (ref: User)",
 "course": "ObjectId (ref: Course)",
 "payment": "ObjectId (ref: Payment)",
 "purchase_date": "ISO Date",
 "access_expires_at": "Date | null",
 "status": "String - 'ACTIVE' | 'EXPIRED' | 'REFUNDED' (default:
'ACTIVE')",
 "createdAt": "ISO Date",
 "updatedAt": "ISO Date"
}
```

```

6.9 Notification

```

```json
{
 "_id": "ObjectId",
 "user": "ObjectId (ref: User)",
 "title": "String (required)",
 "message": "String (required)",
 "type": "String - 'SYSTEM' | 'PURCHASE' | 'COURSE_UPDATE' |
'TEST_RESULT' (default: 'SYSTEM')",

```

```

 "is_read": "Boolean (default: false)",
 "createdAt": "ISO Date",
 "updatedAt": "ISO Date"
 }
 ...

> ⚠ Notifications **auto-delete after 30 days** (TTL index).

```

### ### 6.10 Resource

```

```json
{
  "_id": "ObjectId",
  "title": "String (required, min 3 chars)",
  "description": "String (optional)",
  "course": "ObjectId (ref: Course)",
  "file_url": "String (S3 key)",
  "resource_type": "String - 'pdf' | 'video' | 'notes' | 'assignment' |
'solution'",
  "created_by": "ObjectId (ref: User)",
  "is_active": "Boolean (default: true)",
  "isDeleted": "Boolean (default: false)",
  "deletedAt": "Date | null",
  "createdAt": "ISO Date",
  "updatedAt": "ISO Date"
}
...

```

6.11 Inquiry

```

```json
{
 "_id": "ObjectId",
 "student": "ObjectId (ref: User)",
 "subject": "String (required)",
 "message": "String (required)",
 "status": "String - 'OPEN' | 'IN_PROGRESS' | 'RESOLVED' | 'CLOSED'
(default: 'OPEN')",
 "admin_reply": "String | null",
 "replied_by": "ObjectId (ref: User) | null",

```

```
 "replied_at": "ISO Date | null",
 "createdAt": "ISO Date",
 "updatedAt": "ISO Date"
 }
 ...

```

## ## 7. API Endpoints

### ### 7.1 Health Check

#### `GET /api/v1/healthcheck`

● **\*\*Public\*\***

**\*\*Description:\*\*** Check if the server is running.

**\*\*Response:\*\***

```
```json
{
  "statusCode": 200,
  "data": { "status": "ok" },
  "message": "OK",
  "success": true
}
...

```

7.2 Auth Routes

Base Path: `/api/v1/auth`

`POST /api/v1/auth/register`

****Public****

****Description:**** Register a new user account.

****Request Body (JSON):****

Field	Type	Required	Validation
-----	-----	-----	

`full_name`	String		Min 2 chars, max 50 chars
`email`	String		Must be a valid email
`password`	String		Min 8 chars, must contain at least
one letter and one number			
`phone_number`	String		Optional

****Example Request:****

```
```json
{
 "full_name": "Rahul Sharma",
 "email": "rahul@example.com",
 "password": "SecurePass123",
 "phone_number": "9876543210"
}
```

**\*\*Success Response (201):\*\***

```
```json
{
  "statusCode": 201,
  "data": {
    "_id": "665a...",
    "full_name": "Rahul Sharma",
    "email": "rahul@example.com",
    "phone_number": "9876543210",
    "status": "unverified",
  }
}
```

```
    "role": "STUDENT",
    "createdAt": "2026-06-12T...",
    "updatedAt": "2026-06-12T..."
  },
  "message": "User registered successfully",
  "success": true
}
...

```

****Error Responses:****

- `409` - User with this email already exists
- `400` - Validation error

`POST /api/v1/auth/login`

 ****Public**** (rate limited: 10 attempts/hour)

****Description:**** Login with email and password. Returns tokens in both cookies and response body.

****Request Body (JSON):****

Field	Type	Required	Validation
-----	-----	-----	-----
`email`	String	✓	Must be a valid email
`password`	String	✓	Must not be empty

****Example Request:****

```
```json
{
 "email": "rahul@example.com",
 "password": "SecurePass123"
}
...

```

#### **\*\*Success Response (200):\*\***

```
Sets cookies: `accessToken`, `refreshToken` (httpOnly, secure in
production, sameSite: strict)
```

```
```json
{
  "statusCode": 200,
  "data": {
    "user": {
      "_id": "665a...",
      "full_name": "Rahul Sharma",
      "email": "rahul@example.com",
      "role": "STUDENT",
      "status": "unverified"
    },
    "accessToken": "eyJhbGciOiJIUzI1NiIs... ",
    "refreshToken": "eyJhbGciOiJIUzI1NiIs..."
  },
  "message": "User logged in successfully",
  "success": true
}
```
```

#### **\*\*Error Responses:\*\***

```
- `401` - Invalid credentials
- `403` - Account is temporarily locked (after 5 failed attempts, locked
for 15 minutes)
- `429` - Too many login attempts
```

```
>  **Account Locking:** After **5 consecutive failed login attempts**,
the account is locked for **15 minutes**.
```

---

```
`POST /api/v1/auth/logout`
```

 **\*\*Auth Required\*\***

**\*\*Description:\*\*** Logout current user. Clears refresh token from DB and cookies.

**\*\*Request Body:\*\*** None

**\*\*Headers:\*\***

```

Authorization: Bearer <accessToken>

```

**\*\*Success Response (200):\*\***

```json

```
{
  "statusCode": 200,
  "data": {},
  "message": "User logged out successfully",
  "success": true
}
```

```

---

#### `POST /api/v1/auth/refresh-token`

 **\*\*Public\*\*** (uses refresh token, not access token)

**\*\*Description:\*\*** Get a new pair of access + refresh tokens using the existing refresh token. Implements **\*\*token rotation\*\*** (old refresh token is invalidated).

**\*\*Request Body (JSON):\*\***

Field	Type	Required	Notes
-------	------	----------	-------

--	--	--	--

-----	-----	-----	
-------	-------	-------	--

--	--	--	--

`refreshToken`	String	✗	Can also be sent via `refreshToken` cookie instead
----------------	--------	---	----------------------------------------------------

**\*\*Success Response (200):\*\***

Sets new cookies: `accessToken`, `refreshToken`

```
```json
{
  "statusCode": 200,
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIs... ",
    "refreshToken": "eyJhbGciOiJIUzI1NiIs..."
  },
  "message": "Access token refreshed",
  "success": true
}
```
```

### **\*\*Error Responses:\*\***

- `401` - Invalid/expired refresh token or token reuse detected

>  **\*\*Security:\*\*** If a refresh token is reused (e.g., stolen and used by attacker), the system **\*\*revokes all tokens\*\*** for that user, requiring re-login.

---

### **#### `POST /api/v1/auth/change-password`**

 **\*\*Auth Required\*\***

**\*\*Description:\*\*** Change the password of the currently logged-in user. Invalidates all refresh tokens after change.

### **\*\*Request Body (JSON):\*\***

| Field         | Type   | Required                                                                            | Validation        |
|---------------|--------|-------------------------------------------------------------------------------------|-------------------|
| -----         | -----  | -----                                                                               | -----             |
| `oldPassword` | String |  | Must not be empty |
| `newPassword` | String |  | Min 8 characters  |

### **\*\*Example Request:\*\***

```
```json
{
  "oldPassword": "OldPass123",
  "newPassword": "NewSecure456"
}
```



```

}
...

**Success Response (200):**
```json
{
 "statusCode": 200,
 "data": {},
 "message": "Password changed successfully",
 "success": true
}
...

```

```

Error Responses:
- `400` - Invalid old password

```

```

```

```
`POST /api/v1/auth/forgot-password`
```

 **\*\*Public\*\***

**\*\*Description:\*\*** Request a password reset. Generates a reset token (valid for 10 minutes). Currently mocks email sending – the reset token URL is logged on the server.

**\*\*Request Body (JSON):\*\***

Field	Type	Required	Validation
-----	-----	-----	-----
`email`	String		Must be a valid email

```

Success Response (200):
```json
{
  "statusCode": 200,
  "data": {},
  "message": "Password reset token generated and sent to email",
  "success": true
}

```

****Error Responses:****

- `404` - User with this email does not exist

`POST /api/v1/auth/reset-password/:token`

 ****Public****

****Description:**** Reset password using the token received via email.
Invalidates all refresh tokens after reset.

****URL Params:****

Param	Type	Description
-----	-----	-----
`token`	String	The reset token from the URL

****Request Body (JSON):****

Field	Type	Required	Validation
-----	-----	-----	-----
`password`	String		Min 8 characters

****Success Response (200):****

```
```json
{
 "statusCode": 200,
 "data": {},
 "message": "Password reset successfully",
 "success": true
}
```

\*\*\*

**\*\*Error Responses:\*\***

- `400` - Invalid or expired reset token

---

### ### 7.3 User Routes

Base Path: `/api/v1/users`

> All routes in this section require authentication.

---

#### `GET /api/v1/users/me`

 **\*\*Auth Required\*\***

**\*\*Description:\*\*** Get the current logged-in user's profile.

**\*\*Response (200):\*\***

```
```json
{
  "statusCode": 200,
  "data": {
    "_id": "665a...",
    "full_name": "Rahul Sharma",
    "email": "rahul@example.com",
    "phone_number": "9876543210",
    "profile_picture": null,
    "status": "active",
    "role": "STUDENT",
    "createdAt": "...",
    "updatedAt": "..."
  },
  "message": "User profile fetched successfully",
  "success": true
}
```
```

---

#### `PATCH /api/v1/users/update-account`

 **\*\*Auth Required\*\***

**\*\*Description:\*\*** Update the current user's profile information.

**\*\*Request Body (JSON):\*\***

| Field          | Type   | Required | Validation                          |
|----------------|--------|----------|-------------------------------------|
| -----          | -----  | -----    | -----                               |
| `full_name`    | String | ✗        | Min 2 chars, max 50 chars (if sent) |
| `phone_number` | String | ✗        | Optional                            |

> At least one field must be provided.

**\*\*Success Response (200):\*\*** Returns updated user object.

---

#### `PATCH /api/v1/users/avatar`

🔒 **\*\*Auth Required\*\***

**\*\*Description:\*\*** Update the current user's profile picture.

**\*\*Request Body (JSON):\*\***

| Field             | Type   | Required | Validation    |
|-------------------|--------|----------|---------------|
| -----             | -----  | -----    | -----         |
| `profile_picture` | String | ✓        | Must be a URL |

**\*\*Example Request:\*\***

```
```json
{
  "profile_picture": "https://cloudinary.com/images/avatar123.jpg"
}
```
```

**\*\*Success Response (200):\*\*** Returns updated user object.

---

#### `GET /api/v1/users`

● **\*\*Admin Only\*\***

**\*\*Description:\*\*** Get all users in the system.

**\*\*Response (200):\*\*** Returns array of user objects.

---

#### `GET /api/v1/users/:id`

● **\*\*Admin Only\*\***

**\*\*Description:\*\*** Get a specific user by ID.

**\*\*URL Params:\*\***

| Param | Type   | Description      |  |
|-------|--------|------------------|--|
| ----- | -----  | -----            |  |
| `id`  | String | MongoDB ObjectId |  |

**\*\*Success Response (200):\*\*** Returns single user object.

**\*\*Error:\*\*** `404` – User not found.

---

#### `PATCH /api/v1/users/:id/status`

● **\*\*Admin Only\*\***

**\*\*Description:\*\*** Update a user's account status.

**\*\*URL Params:\*\***

| Param | Type  | Description |  |
|-------|-------|-------------|--|
| ----- | ----- | -----       |  |

|      |        |                  |  |
|------|--------|------------------|--|
| `id` | String | MongoDB ObjectId |  |
|------|--------|------------------|--|

**\*\*Request Body (JSON):\*\***

|          |        |                                                                                   |                                                     |
|----------|--------|-----------------------------------------------------------------------------------|-----------------------------------------------------|
| Field    | Type   | Required                                                                          | Validation                                          |
| -----    | -----  | -----                                                                             |                                                     |
| -----    |        |                                                                                   |                                                     |
| `status` | String |  | Must be one of: `active`, `suspended`, `unverified` |

**\*\*Success Response (200):\*\*** Returns updated user object.

---

#### **DELETE** /api/v1/users/:id`

 **\*\*Admin Only\*\***

**\*\*Description:\*\*** Permanently delete a user.

**\*\*URL Params:\*\***

|       |        |                  |  |
|-------|--------|------------------|--|
| Param | Type   | Description      |  |
| ----- | -----  | -----            |  |
| `id`  | String | MongoDB ObjectId |  |

**\*\*Success Response (200):\*\***

```

```json
{
  "statusCode": 200,
  "data": {},
  "message": "User deleted successfully",
  "success": true
}
```

```

---

### 7.4 Category Routes

Base Path: `/api/v1/categories`

---

#### `GET /api/v1/categories`

● **\*\*Public\*\***

**\*\*Description:\*\*** Get all categories.

**\*\*Response (200):\*\***

```json

```
{
  "statusCode": 200,
  "data": [
    {
      "_id": "665a...",
      "name": "UPSC",
      "description": "UPSC exam preparation",
      "createdAt": "...",
      "updatedAt": "..."
    }
  ],
  "message": "Categories fetched successfully",
  "success": true
}
```

```

---

#### `GET /api/v1/categories/:id`

● **\*\*Public\*\***

**\*\*Description:\*\*** Get a single category by ID.

**\*\*Error:\*\*** `404` – Category not found.

---

#### `POST /api/v1/categories`

● **\*\*Admin Only\*\***

**\*\*Description:\*\*** Create a new category.

**\*\*Request Body (JSON):\*\***

Field	Type	Required	Validation
-----	-----	-----	-----
`name`	String	✓	Min 2 chars, max 100 chars, unique
`description`	String	✗	Optional

**\*\*Success Response (201):\*\*** Returns created category object.

**\*\*Error:\*\*** `400` – Category already exists.

---

#### `PUT /api/v1/categories/:id`

● **\*\*Admin Only\*\***

**\*\*Description:\*\*** Update a category.

**\*\*Request Body (JSON):\*\***

Field	Type	Required	Validation
-----	-----	-----	-----
`name`	String	✗	Min 2 chars, max 100 chars
`description`	String	✗	Optional

**\*\*Success Response (200):\*\*** Returns updated category object.

---

#### `DELETE /api/v1/categories/:id`

● **\*\*Admin Only\*\***



**\*\*Description:\*\*** Delete a category.

**\*\*Success Response (200):\*\*** Returns empty `data`.

---

### ### 7.5 Course Routes

Base Path: `/api/v1/courses`

> All routes require authentication.

---

#### #### `GET /api/v1/courses`



**\*\*Auth Required\*\***

**\*\*Description:\*\*** Get all courses. Students only see active (`is\_active: true`) courses. Admins see all non-deleted courses.

**\*\*Response (200):\*\***

```json

```
{
  "statusCode": 200,
  "data": [
    {
      "_id": "665a...",
      "title": "UPSC Prelims 2026",
      "description": "Comprehensive course...",
      "price": 999,
      "access_type": "paid",
      "category": { "_id": "...", "name": "UPSC" },
      "created_by": "...",
      "is_active": true,
      "isDeleted": false,
      "createdAt": "...",
      "updatedAt": "..."
    }
  ],
}
```

```
"message": "Courses fetched successfully",
"success": true
}
```

`GET /api/v1/courses/:id`

🔒 **Auth Required**

Description: Get a single course by ID. Students can only see active courses.

Error: `404` – Course not found.

`GET /api/v1/courses/my/enrolled`

🔒 **Auth Required**

Description: Get all courses the current student is enrolled in.

Response (200): Returns array of course objects (populated with category).

`POST /api/v1/courses/:id/enroll`

🔒 **Auth Required** (Students only for free courses)

Description: Enroll in a free course. Paid courses must be purchased through the payment flow.

Request Body: None

Success Response (201):
```json
```

```
{
  "statusCode": 201,
  "data": {
    "_id": "...",
    "user": "...",
    "course": "...",
    "status": "ACTIVE"
  },
  "message": "Successfully enrolled in course",
  "success": true
}
...

```

****Error Responses:****

- `400` - Paid courses require purchase
- `400` - Already enrolled
- `404` - Course not found or inactive

`POST /api/v1/courses`

****Admin Only****

****Description:**** Create a new course.

****Request Body (JSON):****

Field	Type	Required	Validation
-----	-----	-----	-----
`title`	String	✓	Min 3 chars
`description`	String	✓	Min 10 chars
`price`	Number	✗	>= 0, default: 0
`access_type`	String	✗	`"free"` or `"paid"`, default: `"paid"`

```
| `category`      | String |  | Valid 24-char hex MongoDB ObjectId  
string          |
```

****Success Response (201):**** Returns created course object.

`PUT /api/v1/courses/:id`

 ****Admin Only****

****Description:**** Update a course. Accepts any subset of the course fields.

****Request Body (JSON):**** Same fields as create, all optional.

****Success Response (200):**** Returns updated course object.

`DELETE /api/v1/courses/:id`

 ****Admin Only****

****Description:**** Soft-delete a course (sets `isDeleted: true`).

`PATCH /api/v1/courses/:id/publish`

 ****Admin Only****

****Description:**** Set course `is\_active` to `true`.

****Request Body:**** None

****Success Response (200):**** Returns updated course object.

`PATCH /api/v1/courses/:id/unpublish`

🔴 ****Admin Only****

****Description:**** Set course `is\_active` to `false`.

****Request Body:**** None

****Success Response (200):**** Returns updated course object.

7.6 Resource Routes

Base Path: `/api/v1/resources`

> All routes require authentication.

`GET /api/v1/resources/course/:courseId`

🔒 ****Auth Required**** (must be enrolled in the course OR be admin)

****Description:**** Get all resources for a specific course.

****URL Params:****

Param	Type	Description	
-----	-----	-----	
`courseId`	String	MongoDB ObjectId	

****Response (200):**** Returns array of resource objects.

****Errors:****

- `403` - Active enrollment required
- `404` - Course not found

`GET /api/v1/resources/:id/download`

 ****Auth Required**** (must be enrolled OR be admin)

****Description:**** Get a ****signed S3 download URL**** for a resource file. The signed URL is ****time-limited****.

****URL Params:****

Param	Type	Description
-----	-----	-----
`id`	String	Resource ObjectId

****Success Response (200):****

```
```json
{
 "statusCode": 200,
 "data": {
 "downloadUrl":
"https://s3.amazonaws.com/bucket/...?X-Amz-Signature=..."
 },
 "message": "Signed URL generated successfully",
 "success": true
}
```
```

>  The `downloadUrl` is a ****pre-signed S3 URL****. Open it in a browser or use it in `` / `window.open()` for download. It expires after a limited time.

`POST /api/v1/resources`

 ****Admin Only****

****Description:**** Upload a new resource file to a course.

****Content-Type:**** `multipart/form-data`

| Field | Type | Location | Required | Validation |
|-----------------|--------|-----------|----------|---|
| ----- | ----- | ----- | ----- | |
| ----- | | | | |
| `file` | File | Form Data | ✓ | Max 50 MB |
| `title` | String | Form Data | ✓ | Min 3 chars |
| `description` | String | Form Data | ✗ | Optional |
| `course` | String | Form Data | ✓ | Valid MongoDB ObjectId |
| `resource_type` | String | Form Data | ✓ | `"pdf"`, `"video"`,
`"notes"`, `"assignment"`, or `"solution"` |

> ⚠ This is the **only endpoint that uses `multipart/form-data`** instead of JSON. Use `FormData` in the frontend.

Success Response (201): Returns created resource object.

DELETE /api/v1/resources/:id

● **Admin Only**

Description: Delete a resource (removes from S3 and database permanently).

Success Response (200): Returns empty `data`.

7.7 Mock Test Routes

Base Path: `/api/v1/mock-tests`

> All routes require authentication.

`GET /api/v1/mock-tests`

 ****Auth Required****

****Description:**** Get all mock tests. Students only see active tests.

****Correct answers (`is\_correct`) are hidden**** from students.

****Response (200):**** Returns array of mock test objects.

`GET /api/v1/mock-tests/:id`

 ****Auth Required****

****Description:**** Get a single mock test by ID. ****Correct answers are hidden**** for non-admin users.

****Error:**** `404` – Mock Test not found.

`POST /api/v1/mock-tests`

 ****Admin Only****

****Description:**** Create a new mock test.

****Request Body (JSON):****

| Field | Type | Required | Validation / Default |
|---------------|--------|---|----------------------|
| ----- | ----- | ----- | |
| ----- | ----- | | |
| `title` | String |  | Min 3 chars |
| `description` | String |  | Optional |
| | | | |

| | | | |
|--------------------------------|---------|---|------------------------|
| `course` | String | ❌ | Valid MongoDB ObjectId |
| | | | |
| `category` | String | ❌ | Valid MongoDB ObjectId |
| | | | |
| `difficulty` | String | ❌ | `"easy"`, `"medium"`, |
| `"hard"` (default: `"medium"`) | | | |
| `total_questions` | Number | ✅ | >= 1 |
| | | | |
| `passing_marks` | Number | ✅ | >= 1 |
| | | | |
| `negative_marking` | Boolean | ❌ | Default: `false` |
| | | | |
| `negative_marks_per_wrong` | Number | ❌ | >= 0, default: 0 |
| | | | |
| `total_marks` | Number | ✅ | >= 1 |
| | | | |
| `duration_minutes` | Number | ✅ | >= 1 |
| | | | |

****Success Response (201):**** Returns created mock test object.

`PUT /api/v1/mock-tests/:id`

● ****Admin Only****

****Description:**** Update a mock test. All fields are optional (partial update).

`DELETE /api/v1/mock-tests/:id`

● ****Admin Only****

****Description:**** Soft-delete a mock test.

`PATCH /api/v1/mock-tests/:id/publish`

● ****Admin Only****

****Description:**** Publish a mock test (`is\_active: true`).

****Request Body:**** None

`PATCH /api/v1/mock-tests/:id/unpublish`

● ****Admin Only****

****Description:**** Unpublish a mock test (`is\_active: false`).

****Request Body:**** None

`POST /api/v1/mock-tests/:id/questions`

● ****Admin Only****

****Description:**** Add a single question to a mock test.

****Request Body (JSON):****

| Field | Type | Required | Validation |
|---------------|--------|----------|------------------------|
| ----- | ----- | ----- | ----- |
| `text` | String | ✓ | Min 1 char |
| `marks` | Number | ✗ | >= 1, default: 1 |
| `explanation` | String | ✗ | Optional |
| `options` | Array | ✓ | Min 2 options required |

****Each Option:****

| Field | Type | Required | Description |
|--------|--------|----------|-------------------------|
| ----- | ----- | ----- | ----- |
| `text` | String | ✓ | Option text, min 1 char |

| | | | | |
|--------------|---------|---|----------------------------|--|
| `is_correct` | Boolean |  | Whether this is the answer | |
|--------------|---------|---|----------------------------|--|

****Example Request:****

```
```json
{
 "text": "What is the capital of India?",
 "marks": 2,
 "explanation": "New Delhi is the capital of India",
 "options": [
 { "text": "Mumbai", "is_correct": false },
 { "text": "New Delhi", "is_correct": true },
 { "text": "Kolkata", "is_correct": false },
 { "text": "Chennai", "is_correct": false }
]
}
```
```

****Success Response (201):**** Returns the created question object.

`POST /api/v1/mock-tests/:id/questions/bulk`

 ****Admin Only****

****Description:**** Add multiple questions at once.

****Request Body (JSON):****

| | | | | |
|-------------|-------|---|-----------------------------------|--|
| Field | Type | Required | Validation | |
| ----- | ---- | ----- | ----- | |
| `questions` | Array |  | Array of question objects (min 1) | |

Each question follows the same format as the single question endpoint above.

****Success Response (201):**** Returns the updated mock test object with all questions.

`PUT /api/v1/mock-tests/:id/questions/:questionId`

 ****Admin Only****

****Description:**** Update a specific question.

****URL Params:****

| Param | Type | Description |
|--------------|--------|--------------------|
| ----- | ----- | ----- |
| `id` | String | Mock Test ObjectId |
| `questionId` | String | Question ObjectId |

****Request Body:**** Same format as add question.

`DELETE /api/v1/mock-tests/:id/questions/:questionId`

 ****Admin Only****

****Description:**** Delete a specific question from a mock test.

7.8 Test Attempt Routes

Base Path: `/api/v1/attempts`

> All routes require authentication.

`POST /api/v1/attempts/start`

 ****Auth Required****

****Description:**** Start a new test attempt. If an `IN\_PROGRESS` attempt already exists for the same test, it will be returned instead of creating

a new one. A ****BullMQ auto-submit job**** is scheduled to automatically submit the test when the duration expires.

****Request Body (JSON):****

| Field | Type | Required | Validation |
|----------------|--------|----------|------------------------|
| ----- | ----- | ----- | ----- |
| `mock_test_id` | String | ✓ | Valid MongoDB ObjectId |

****Example Request:****

```
```json
{
 "mock_test_id": "665a1234abcd5678ef901234"
}
```
```

****Success Response (201):****

```
```json
{
 "statusCode": 201,
 "data": {
 "_id": "...",
 "user": "...",
 "mock_test": "665a1234abcd5678ef901234",
 "started_at": "2026-06-12T...",
 "status": "IN_PROGRESS",
 "answers": [],
 "score": 0,
 "percentage": 0
 },
 "message": "Test started successfully",
 "success": true
}
```
```

****Error Responses:****

- `400` - Maximum attempt limit reached (max 3 attempts per test)
- `404` - Mock test not found or inactive

> ⚠️ ****Max 3 attempts**** allowed per user per mock test.

`PUT /api/v1/attempts/:attemptId/answer`

 ****Auth Required****

****Description:**** Save or update an answer for a question in an active test attempt. Can also mark a question for review.

****URL Params:****

| Param | Type | Description |
|-------------|--------|----------------------|
| ----- | ----- | ----- |
| `attemptId` | String | TestAttempt ObjectId |

****Request Body (JSON):****

| Field | Type | Required | Validation |
|------------------------|---------|----------|--|
| ----- | ----- | ----- | ----- |
| ----- | | | |
| `question_id` | String | ✓ | Valid MongoDB ObjectId |
| `selected_option_id` | String | ✗ | Valid MongoDB ObjectId or `null` (to deselect) |
| `is_marked_for_review` | Boolean | ✗ | Default: `false` |

****Example Request:****

```
```json
{
 "question_id": "665a...",
 "selected_option_id": "665b...",
 "is_marked_for_review": false
}
```
```

****Success Response (200):**** Returns the updated `answers` array.

****Error Responses:****

- `404` - Active test attempt not found
- `404` - Question not found in this test
- `404` - Option not found

`POST /api/v1/attempts/:attemptId/submit`

 ****Auth Required****

****Description:**** Manually submit the test. Changes status from `IN\_PROGRESS` to `COMPLETED`.

****URL Params:****

| Param | Type | Description |
|-------------|--------|----------------------|
| ----- | ----- | ----- |
| `attemptId` | String | TestAttempt ObjectId |

****Request Body:**** None

****Success Response (200):****

```
```json
{
 "statusCode": 200,
 "data": {
 "_id": "...",
 "status": "COMPLETED",
 "completed_at": "2026-06-12T...",
 "answers": [...],
 "score": 0,
 "percentage": 0
 },
 "message": "Test submitted successfully",
 "success": true
}
```
```

>  Tests are also ****auto-submitted**** when the duration expires (via background BullMQ job).

`POST /api/v1/attempts/:attemptId/evaluate`

 ****Auth Required****

****Description:**** Evaluate/grade a completed test attempt. Calculates score based on correct answers, applies negative marking if enabled.

****URL Params:****

| Param | Type | Description |
|-------------|--------|----------------------|
| ----- | ----- | ----- |
| `attemptId` | String | TestAttempt ObjectId |

****Request Body:**** None

****Success Response (200):****

```json

```
{
 "statusCode": 200,
 "data": {
 "_id": "...",
 "status": "COMPLETED",
 "answers": [
 {
 "question_id": "...",
 "question_text": "What is the capital of India?",
 "selected_option_id": "...",
 "selected_option_text": "New Delhi",
 "is_correct": true,
 "is_marked_for_review": false,
 "answered_at": "..."
 }
],
 "score": 18,
 "percentage": 90
 }
}
```



```
 },
 "message": "Test evaluated successfully",
 "success": true
 }
  ````
```

****Error Responses:****

- `400` - Test is not completed yet
- `404` - Attempt not found

> ⚠️ **Scoring Logic:**

- > - Correct answer: `+question.marks`
- > - Wrong answer (if `negative\_marking: true`):
`-mockTest.negative\_marks\_per\_wrong`
- > - Unanswered: `0`
- > - Score is clamped to minimum `0` (never negative)

`GET /api/v1/attempts/:attemptId`

🔒 ****Auth Required****

****Description:**** Get a specific test attempt by ID.

****Success Response (200):**** Returns the full test attempt object.

7.9 Payment Routes (Razorpay)

Base Path: `/api/v1/payments`

> All routes require authentication.

`POST /api/v1/payments/create-order`

🔒 ****Auth Required****

****Description:**** Create a Razorpay order for purchasing a course or item.

****Request Body (JSON):****

| Field | Type | Required | Description |
|-------------|--------|----------|--|
| ----- | ----- | ----- | |
| ----- | | | |
| `item_id` | String | ✓ | MongoDB ObjectId of the item being purchased |
| `item_type` | String | ✓ | Type of item (e.g., `"Course"`) |
| `amount` | Number | ✓ | Amount in **INR** (rupees, NOT paise) |

****Example Request:****

```
```json
{
 "item_id": "665a...",
 "item_type": "Course",
 "amount": 999
}
```

**\*\*Success Response (200):\*\***

```
```json
{
  "statusCode": 200,
  "data": {
    "id": "order_NxP...",
    "entity": "order",
    "amount": 99900,
    "amount_paid": 0,
    "amount_due": 99900,
    "currency": "INR",
    "receipt": "receipt_order_1718...",
    "status": "created"
  },
  "message": "Order created",
}
```

```

    "success": true
  }
  ...

> ⚠ The `data.id` (Razorpay Order ID) is needed for opening the Razorpay
checkout modal.
> ⚠ The `data.amount` in the response is in **paise** (amount × 100).

---

#### `POST /api/v1/payments/verify`

🔒 **Auth Required**

**Description:** Verify Razorpay payment after the user completes payment
in the Razorpay modal. On success, automatically creates a **Purchase**
record and auto-enrolls the user if the item is a Course.

**Request Body (JSON):**

| Field | Type | Required | Description |
| --- | --- | --- | --- |
| `razorpay_order_id` | String | ✓ | The order ID from
create-order response |
| `razorpay_payment_id` | String | ✓ | From Razorpay success
callback |
| `razorpay_signature` | String | ✓ | From Razorpay success
callback |

**Example Request:**
```json
{
 "razorpay_order_id": "order_NxP...",
 "razorpay_payment_id": "pay_NxQ...",
 "razorpay_signature": "a1b2c3d4e5..."
}
...

```

## **\*\*Success Response (200):\*\***

```
```json
{
  "statusCode": 200,
  "data": {
    "_id": "...",
    "user": "...",
    "order_id": "order_NxP...",
    "amount": 999,
    "currency": "INR",
    "payment_status": "SUCCESS",
    "payment_id": "pay_NxQ..."
  },
  "message": "Payment verified successfully",
  "success": true
}
```
```

## **\*\*Error Responses:\*\***

- `400` - Payment verification failed (signature mismatch)
- `404` - Order not found

## **> ⚠️ \*\*Frontend Integration Flow:\*\***

- > 1. Call `POST /payments/create-order` to get a Razorpay order ID
- > 2. Open Razorpay checkout modal with the order ID
- > 3. On success callback, send the 3 Razorpay values to `POST /payments/verify`
- > 4. On success, the user is auto-enrolled in the course

---

## **### 7.10 Notification Routes**

Base Path: `/api/v1/notifications`

- > All routes require authentication.

---

**#### `GET /api/v1/notifications`**

 **\*\*Auth Required\*\***

**\*\*Description:\*\*** Get all notifications for the current user, sorted by most recent first.

**\*\*Response (200):\*\***

```
```json
{
  "statusCode": 200,
  "data": [
    {
      "_id": "...",
      "user": "...",
      "title": "Course Enrolled",
      "message": "You've been enrolled in UPSC Prelims 2026",
      "type": "PURCHASE",
      "is_read": false,
      "createdAt": "..."
    }
  ],
  "message": "Notifications fetched",
  "success": true
}
```
```

----

#### `PATCH /api/v1/notifications/:id/read`

 **\*\*Auth Required\*\***

**\*\*Description:\*\*** Mark a notification as read.

**\*\*URL Params:\*\***

| Param | Type   | Description           |
|-------|--------|-----------------------|
| ----- | -----  | -----                 |
| `id`  | String | Notification ObjectId |

**\*\*Request Body:\*\*** None

**\*\*Success Response (200):\*\*** Returns the updated notification with  
`is\_read: true`.

---

### ### 7.11 Dashboard Routes

Base Path: `/api/v1/dashboard`

> All routes require authentication.

---

#### `GET /api/v1/dashboard/student`

 **\*\*Auth Required\*\***

**\*\*Description:\*\*** Get the student's dashboard statistics.

**\*\*Response (200):\*\***

```json

```
{
  "statusCode": 200,
  "data": {
    "totalAttempts": 15,
    "avgScore": "78.50",
    "recentActivity": [
      {
        "_id": "...",
        "mock_test": "...",
        "score": 85,
        "percentage": 90,
        "status": "COMPLETED",
        "completed_at": "..."
      }
    ]
  },
  "message": "Student dashboard fetched",
```

```
    "success": true
  }
  ...

> `avgScore` is returned as a string (formatted to 2 decimal places).
> `recentActivity` contains the last 5 completed test attempts.
```

`GET /api/v1/dashboard/admin``

 **Admin Only**

Description: Get the admin dashboard statistics.

Response (200):

```
```json
{
 "statusCode": 200,
 "data": {
 "totalStudents": 250,
 "totalCourses": 12,
 "totalTests": 45,
 "revenue": 125000
 },
 "message": "Admin dashboard fetched",
 "success": true
}
```
```

> `revenue` is the total amount (in INR) from all successful payments.

7.12 Leaderboard Routes

Base Path: `/api/v1/leaderboard``

> All routes require authentication.

```

---

#### `GET /api/v1/leaderboard/:testId`

🔒 **Auth Required**

**Description:** Get the top 10 leaderboard for a specific mock test.
Sorted by highest score, then earliest completion. Results are **cached in
Redis for 5 minutes**.

**URL Params:**

| Param      | Type   | Description           |
| ----- | ----- | ----- |
| `testId` | String | MockTest ObjectId |

**Response (200):**
```json
{
 "statusCode": 200,
 "data": [
 {
 "_id": "...",
 "user": { "_id": "...", "name": "...", "email": "..." },
 "mock_test": "...",
 "score": 95,
 "percentage": 100,
 "completed_at": "..."
 }
],
 "message": "Leaderboard fetched",
 "success": true
}
```

---

### 7.13 Inquiry Routes

Base Path: `/api/v1/inquiries`

```


> All routes require authentication.

`POST /api/v1/inquiries`

 **Auth Required** (Students)

Description: Submit a new inquiry/support ticket.

Request Body (JSON):

| Field | Type | Required | Description |
|------------------------|--------|----------|------------------------|
| ----- | ----- | ----- | ----- |
| <code>`subject`</code> | String | ✓ | Subject of the inquiry |
| <code>`message`</code> | String | ✓ | Detailed message |

Example Request:

```
```json
{
 "subject": "Unable to access course materials",
 "message": "I purchased the UPSC course but cannot see the study
materials."
}
```
```

Success Response (201):

```
```json
{
 "statusCode": 201,
 "data": {
 "_id": "...",
 "student": "...",
 "subject": "Unable to access course materials",
 "message": "I purchased the UPSC course...",
 "status": "OPEN",
 "createdAt": "..."
 },
 "message": "Inquiry submitted",
}
```

```
"success": true
}
...

`GET /api/v1/inquiries/my`

🔒 **Auth Required** (Students)

Description: Get all inquiries submitted by the current user.

Response (200): Returns array of inquiry objects sorted by most recent.

`GET /api/v1/inquiries`

🔴 **Admin Only**

Description: Get all inquiries from all students. Populated with student name and email.

Response (200):
```json
{
  "statusCode": 200,
  "data": [
    {
      "_id": "...",
      "student": { "_id": "...", "name": "...", "email": "..." },
      "subject": "...",
      "message": "...",
      "status": "OPEN",
      "createdAt": "..."
    }
  ],
  "message": "All inquiries",
  "success": true
}
```

```

}
...

---

#### `PATCH /api/v1/inquiries/:id/reply`

🔴 **Admin Only**

**Description:** Reply to a student inquiry. Sets status to `RESOLVED`.

**URL Params:**

| Param | Type   | Description      |
|-------|--------|------------------|
| ----- | -----  | -----            |
| `id`  | String | Inquiry ObjectId |

**Request Body (JSON):**

| Field   | Type   | Required | Description   |
|---------|--------|----------|---------------|
| -----   | -----  | -----    | -----         |
| `reply` | String | ✓        | Admin's reply |

**Example Request:**

```

```json
{
  "reply": "We've fixed the issue. Please try accessing the materials
again."
}
```

```

**Success Response (200):**

```

```json
{
  "statusCode": 200,
  "data": {
    "_id": "...",
    "status": "RESOLVED",
    "admin_reply": "We've fixed the issue...",
    "replied_by": "...",

```


```

```
 "replied_at": "2026-06-12T..."
 },
 "message": "Inquiry replied",
 "success": true
}
...

```

## ## 8. Enum Values Reference

| Model        | Field           | Valid Values                                            |
|--------------|-----------------|---------------------------------------------------------|
| -----        |                 |                                                         |
| User         | `role`          | `STUDENT`, `ADMIN`,<br>`INSTRUCTOR`                     |
| User         | `status`        | `active`, `suspended`,<br>`unverified`                  |
| Course       | `access_type`   | `free`, `paid`                                          |
| MockTest     | `difficulty`    | `easy`, `medium`, `hard`                                |
| Resource     | `resource_type` | `pdf`, `video`, `notes`,<br>`assignment`, `solution`    |
| Enrollment   | `status`        | `ACTIVE`, `EXPIRED`,<br>`REVOKED`                       |
| TestAttempt  | `status`        | `IN_PROGRESS`, `COMPLETED`,<br>`ABANDONED`              |
| Payment      | `status`        | `PENDING`, `SUCCESS`,<br>`FAILED`, `REFUNDED`           |
| Purchase     | `status`        | `ACTIVE`, `EXPIRED`,<br>`REFUNDED`                      |
| Notification | `type`          | `SYSTEM`, `PURCHASE`,<br>`COURSE_UPDATE`, `TEST_RESULT` |
| Inquiry      | `status`        | `OPEN`, `IN_PROGRESS`,<br>`RESOLVED`, `CLOSED`          |
| -----        |                 |                                                         |

## ## 9. Important Notes for Frontend

### ### 9.1 Cookie-Based Auth with CORS

When making API calls from the frontend, you **must** set ``withCredentials: true`` in your HTTP client (Axios, Fetch, etc.):

```
```javascript
// Axios example
axios.defaults.withCredentials = true;
axios.defaults.baseURL = 'http://localhost:3000/api/v1';

// Or per-request
axios.get('/users/me', { withCredentials: true });

// Fetch example
fetch('http://localhost:3000/api/v1/users/me', {
  credentials: 'include',
  headers: {
    'Authorization': `Bearer ${accessToken}`
  }
});
```
```

### ### 9.2 Token Refresh Flow

Implement an **Axios interceptor** (or equivalent) to handle token refresh:

1. If a request returns ``401``, call ``POST /auth/refresh-token``
2. If refresh succeeds, retry the original request with the new token
3. If refresh fails, redirect to login page

### ### 9.3 File Upload (Resource Upload)

Use ``FormData`` for the resource upload endpoint:

```
```javascript
const formData = new FormData();
formData.append('file', fileInput.files[0]);
```

```

formData.append('title', 'Study Notes Chapter 1');
formData.append('course', '665a...');
formData.append('resource_type', 'pdf');

axios.post('/resources', formData, {
  headers: { 'Content-Type': 'multipart/form-data' }
});
```

```

### ### 9.4 Razorpay Payment Integration

```

```javascript
// Step 1: Create order
const { data } = await axios.post('/payments/create-order', {
  item_id: courseId,
  item_type: 'Course',
  amount: 999
});

// Step 2: Open Razorpay checkout
const options = {
  key: 'YOUR_RAZORPAY_KEY_ID', // Get from backend team
  amount: data.data.amount,
  currency: data.data.currency,
  order_id: data.data.id,
  handler: async function (response) {
    // Step 3: Verify payment
    await axios.post('/payments/verify', {
      razorpay_order_id: response.razorpay_order_id,
      razorpay_payment_id: response.razorpay_payment_id,
      razorpay_signature: response.razorpay_signature
    });
    // Payment successful - user is now enrolled
  }
};

const rzp = new Razorpay(options);
rzp.open();
```

```

### ### 9.5 Test-Taking Flow

1. **\*\*Start Test:\*\*** `POST /attempts/start` → save the returned `attemptId`
2. **\*\*Save Answers:\*\*** `PUT /attempts/:attemptId/answer` (call on each answer)
3. **\*\*Submit Test:\*\*** `POST /attempts/:attemptId/submit` (manual submit)
4. **\*\*Get Results:\*\*** `POST /attempts/:attemptId/evaluate` (grades the test)
5. **\*\*View Results:\*\*** `GET /attempts/:attemptId` (fetch attempt details)

> Tests auto-submit after the duration expires. Build a countdown timer on the frontend.

### ### 9.6 Soft Deletes

Courses, MockTests, and Resources use **\*\*soft deletes\*\*** (`isDeleted: true`). They won't appear in list endpoints after deletion but are not permanently removed from the database.

### ### 9.7 ObjectId Format

All MongoDB ObjectId fields are **\*\*24-character hex strings\*\***. Example: `"665a1234abcd5678ef901234"`. Use this regex for frontend validation: `/^[0-9a-fA-F]{24}\$/`

### ### 9.8 Timestamps

All models include `createdAt` and `updatedAt` fields in **\*\*ISO 8601\*\*** format. Example: `"2026-06-12T14:30:00.000Z"`

---

### ## Complete Routes Summary Table

| Method       | Endpoint                | Access   |
|--------------|-------------------------|----------|
| Content-Type |                         |          |
| -----        | -----                   |          |
| -----        | -----                   |          |
| `GET`        | `/api/v1/healthcheck`   | ● Public |
| JSON         |                         |          |
| `POST`       | `/api/v1/auth/register` | ● Public |
| JSON         |                         |          |

|               |                                      |                                                                                              |
|---------------|--------------------------------------|----------------------------------------------------------------------------------------------|
| `POST`        | `/api/v1/auth/login`                 |  Public   |
| JSON          |                                      |                                                                                              |
| `POST`        | `/api/v1/auth/logout`                |  Auth     |
| -             |                                      |                                                                                              |
| `POST`        | `/api/v1/auth/refresh-token`         |  Public   |
| JSON / Cookie |                                      |                                                                                              |
| `POST`        | `/api/v1/auth/change-password`       |  Auth     |
| JSON          |                                      |                                                                                              |
| `POST`        | `/api/v1/auth/forgot-password`       |  Public   |
| JSON          |                                      |                                                                                              |
| `POST`        | `/api/v1/auth/reset-password/:token` |  Public   |
| JSON          |                                      |                                                                                              |
| `GET`         | `/api/v1/users/me`                   |  Auth     |
| -             |                                      |                                                                                              |
| `PATCH`       | `/api/v1/users/update-account`       |  Auth     |
| JSON          |                                      |                                                                                              |
| `PATCH`       | `/api/v1/users/avatar`               |  Auth     |
| JSON          |                                      |                                                                                              |
| `GET`         | `/api/v1/users`                      |  Admin    |
| -             |                                      |                                                                                              |
| `GET`         | `/api/v1/users/:id`                  |  Admin  |
| -             |                                      |                                                                                              |
| `PATCH`       | `/api/v1/users/:id/status`           |  Admin  |
| JSON          |                                      |                                                                                              |
| `DELETE`      | `/api/v1/users/:id`                  |  Admin  |
| -             |                                      |                                                                                              |
| `GET`         | `/api/v1/categories`                 |  Public |
| -             |                                      |                                                                                              |
| `GET`         | `/api/v1/categories/:id`             |  Public |
| -             |                                      |                                                                                              |
| `POST`        | `/api/v1/categories`                 |  Admin  |
| JSON          |                                      |                                                                                              |
| `PUT`         | `/api/v1/categories/:id`             |  Admin  |
| JSON          |                                      |                                                                                              |
| `DELETE`      | `/api/v1/categories/:id`             |  Admin  |
| -             |                                      |                                                                                              |
| `GET`         | `/api/v1/courses`                    |  Auth   |
| -             |                                      |                                                                                              |
| `GET`         | `/api/v1/courses/:id`                |  Auth   |
| -             |                                      |                                                                                              |



|                     |                                         |                                                                                             |
|---------------------|-----------------------------------------|---------------------------------------------------------------------------------------------|
| `GET`               | `/api/v1/courses/my/enrolled`           |  Auth    |
| -                   |                                         |                                                                                             |
| `POST`              | `/api/v1/courses/:id/enroll`            |  Auth    |
| -                   |                                         |                                                                                             |
| `POST`              | `/api/v1/courses`                       |  Admin   |
| JSON                |                                         |                                                                                             |
| `PUT`               | `/api/v1/courses/:id`                   |  Admin   |
| JSON                |                                         |                                                                                             |
| `DELETE`            | `/api/v1/courses/:id`                   |  Admin   |
| -                   |                                         |                                                                                             |
| `PATCH`             | `/api/v1/courses/:id/publish`           |  Admin   |
| -                   |                                         |                                                                                             |
| `PATCH`             | `/api/v1/courses/:id/unpublish`         |  Admin   |
| -                   |                                         |                                                                                             |
| `GET`               | `/api/v1/resources/course/:courseId`    |  Auth    |
| -                   |                                         |                                                                                             |
| `GET`               | `/api/v1/resources/:id/download`        |  Auth    |
| -                   |                                         |                                                                                             |
| `POST`              | `/api/v1/resources`                     |  Admin   |
| multipart/form-data |                                         |                                                                                             |
| `DELETE`            | `/api/v1/resources/:id`                 |  Admin |
| -                   |                                         |                                                                                             |
| `GET`               | `/api/v1/mock-tests`                    |  Auth  |
| -                   |                                         |                                                                                             |
| `GET`               | `/api/v1/mock-tests/:id`                |  Auth  |
| -                   |                                         |                                                                                             |
| `POST`              | `/api/v1/mock-tests`                    |  Admin |
| JSON                |                                         |                                                                                             |
| `PUT`               | `/api/v1/mock-tests/:id`                |  Admin |
| JSON                |                                         |                                                                                             |
| `DELETE`            | `/api/v1/mock-tests/:id`                |  Admin |
| -                   |                                         |                                                                                             |
| `PATCH`             | `/api/v1/mock-tests/:id/publish`        |  Admin |
| -                   |                                         |                                                                                             |
| `PATCH`             | `/api/v1/mock-tests/:id/unpublish`      |  Admin |
| -                   |                                         |                                                                                             |
| `POST`              | `/api/v1/mock-tests/:id/questions`      |  Admin |
| JSON                |                                         |                                                                                             |
| `POST`              | `/api/v1/mock-tests/:id/questions/bulk` |  Admin |
| JSON                |                                         |                                                                                             |

|          |                                                |                                                                                             |
|----------|------------------------------------------------|---------------------------------------------------------------------------------------------|
| `PUT`    | `/api/v1/mock-tests/:id/questions/:questionId` |  Admin   |
| JSON     |                                                |                                                                                             |
| `DELETE` | `/api/v1/mock-tests/:id/questions/:questionId` |  Admin   |
| -        |                                                |                                                                                             |
| `POST`   | `/api/v1/attempts/start`                       |  Auth    |
| JSON     |                                                |                                                                                             |
| `PUT`    | `/api/v1/attempts/:attemptId/answer`           |  Auth    |
| JSON     |                                                |                                                                                             |
| `POST`   | `/api/v1/attempts/:attemptId/submit`           |  Auth    |
| -        |                                                |                                                                                             |
| `POST`   | `/api/v1/attempts/:attemptId/evaluate`         |  Auth    |
| -        |                                                |                                                                                             |
| `GET`    | `/api/v1/attempts/:attemptId`                  |  Auth    |
| -        |                                                |                                                                                             |
| `POST`   | `/api/v1/payments/create-order`                |  Auth    |
| JSON     |                                                |                                                                                             |
| `POST`   | `/api/v1/payments/verify`                      |  Auth    |
| JSON     |                                                |                                                                                             |
| `GET`    | `/api/v1/notifications`                        |  Auth    |
| -        |                                                |                                                                                             |
| `PATCH`  | `/api/v1/notifications/:id/read`               |  Auth  |
| -        |                                                |                                                                                             |
| `GET`    | `/api/v1/dashboard/student`                    |  Auth  |
| -        |                                                |                                                                                             |
| `GET`    | `/api/v1/dashboard/admin`                      |  Admin |
| -        |                                                |                                                                                             |
| `GET`    | `/api/v1/leaderboard/:testId`                  |  Auth  |
| -        |                                                |                                                                                             |
| `POST`   | `/api/v1/inquiries`                            |  Auth  |
| JSON     |                                                |                                                                                             |
| `GET`    | `/api/v1/inquiries/my`                         |  Auth  |
| -        |                                                |                                                                                             |
| `GET`    | `/api/v1/inquiries`                            |  Admin |
| -        |                                                |                                                                                             |
| `PATCH`  | `/api/v1/inquiries/:id/reply`                  |  Admin |
| JSON     |                                                |                                                                                             |

---

**\*\*Total Routes: 47\*\***

*\*Document generated from source code analysis of mastermock\_backend.\**

## V# 🚀 MasterMock Backend – Complete API Documentation

**\*\*Version:\*\*** 1.0

**\*\*Base URL:\*\*** `http://localhost:3000/api/v1`

**\*\*Last Updated:\*\*** June 13, 2026

---

### ## 📄 Table of Contents

1. [General Information] ([#1-general-information](#))
2. [Authentication & Authorization] ([#2-authentication--authorization](#))
3. [Standard Response Format] ([#3-standard-response-format](#))
4. [Standard Error Format] ([#4-standard-error-format](#))
5. [Rate Limiting] ([#5-rate-limiting](#))
6. [Data Models (Schemas)] ([#6-data-models-schemas](#))
7. [API Endpoints] ([#7-api-endpoints](#))
  - [Health Check] ([#71-health-check](#))
  - [Auth Routes] ([#72-auth-routes](#))
  - [User Routes] ([#73-user-routes](#))
  - [Category Routes] ([#74-category-routes](#))
  - [Course Routes] ([#75-course-routes](#))
  - [Resource Routes] ([#76-resource-routes](#))
  - [Mock Test Routes] ([#77-mock-test-routes](#))
  - [Test Attempt Routes] ([#78-test-attempt-routes](#))
  - [Payment Routes] ([#79-payment-routes](#))
  - [Notification Routes] ([#710-notification-routes](#))
  - [Dashboard Routes] ([#711-dashboard-routes](#))
  - [Leaderboard Routes] ([#712-leaderboard-routes](#))
  - [Inquiry Routes] ([#713-inquiry-routes](#))
8. [Enum Values Reference] ([#8-enum-values-reference](#))
9. [Important Notes for Frontend] ([#9-important-notes-for-frontend](#))

---

### ## 1. General Information

| Property                   | Value                                          |  |
|----------------------------|------------------------------------------------|--|
| -----                      | -----                                          |  |
| <b>**Framework**</b>       | Express.js (Node.js)                           |  |
| <b>**Database**</b>        | MongoDB (Mongoose ODM)                         |  |
| <b>**Content-Type**</b>    | `application/json` (all requests/responses)    |  |
| <b>**File Uploads**</b>    | `multipart/form-data` (resource upload only)   |  |
| <b>**Credentials**</b>     | Cookies are used, set `withCredentials: true`  |  |
| <b>**Body Size Limit**</b> | 16 KB (JSON/URL-encoded), 50 MB (file uploads) |  |
| <b>**Payment Gateway**</b> | Razorpay                                       |  |

---

## ## 2. Authentication & Authorization

### ### How Authentication Works

The API uses **\*\*JWT (JSON Web Token)\*\*** for authentication with a **\*\*dual-token system\*\***:

- **\*\*Access Token\*\*** – Short-lived (default: 1 day). Used for API requests.
- **\*\*Refresh Token\*\*** – Long-lived (default: 10 days). Used to get new access tokens.

### ### Sending the Access Token

The token can be sent in **\*\*one of two ways\*\*** (backend checks both):

```

Option 1: Authorization Header (RECOMMENDED)

Authorization: Bearer <accessToken>

Option 2: Cookies (set automatically on login)

Cookie: accessToken=<accessToken>; refreshToken=<refreshToken>

```

### ### User Roles

| Role      | Description                                      |  |
|-----------|--------------------------------------------------|--|
| -----     | -----                                            |  |
| `STUDENT` | Default role. Can view courses, take tests, etc. |  |

|              |                                                |  |
|--------------|------------------------------------------------|--|
| `ADMIN`      | Full access. Can manage courses, tests, users. |  |
| `INSTRUCTOR` | Reserved for future use.                       |  |

### ### Access Markers Used in This Document

|                                                                                                            |                                        |  |
|------------------------------------------------------------------------------------------------------------|----------------------------------------|--|
| Marker                                                                                                     | Meaning                                |  |
| -----                                                                                                      | -----                                  |  |
|  <b>**Public**</b>        | No authentication required             |  |
|  <b>**Auth Required**</b> | Must send a valid access token         |  |
|  <b>**Admin Only**</b>    | Must be logged in with `role: "ADMIN"` |  |

---

## ## 3. Standard Response Format

**\*\*All successful responses\*\*** follow this JSON structure:

```
```json
```

```
{
  "statusCode": 200,
  "data": { ... },
  "message": "Success message here",
  "success": true
}
```

```
```
```

|              |         |                                            |  |
|--------------|---------|--------------------------------------------|--|
| Field        | Type    | Description                                |  |
| -----        | -----   | -----                                      |  |
| `statusCode` | Number  | HTTP status code (200, 201, etc.)          |  |
| `data`       | Any     | The response payload (object, array, etc.) |  |
| `message`    | String  | Human-readable success message             |  |
| `success`    | Boolean | Always `true` for status < 400             |  |

---

## ## 4. Standard Error Format

**\*\*All error responses\*\*** follow this JSON structure:

```
```json
```

```
{
  "success": false,
  "message": "Error message here",
  "errors": [],
  "stack": "..." // Only in development environment
}
```

...

| Field | Type | Description |
|-----------|---------|--|
| ----- | ----- | |
| `success` | Boolean | Always `false` |
| `message` | String | Human-readable error message |
| `errors` | Array | Validation errors array (from Zod validation) |
| `stack` | String | Stack trace (only present in development mode) |

Common HTTP Error Codes

| Code | Meaning |
|-------|---|
| ----- | ----- |
| `400` | Bad Request / Validation Error |
| `401` | Unauthorized (no/invalid token) |
| `403` | Forbidden (wrong role / suspended / account locked) |
| `404` | Not Found |
| `409` | Conflict (duplicate entry) |
| `429` | Too Many Requests (rate limited) |
| `500` | Internal Server Error |

5. Rate Limiting

| Limiter | Scope | Limit |
|---------|-------|-------|
| | | |

| | | | |
|---------------------------|----------------------------------|----------------------------|--|
| | ----- | ----- | |
| | ----- | | |
| **Global Limiter** | All <code>`/api/*`</code> routes | 100 requests per 15 | |
| minutes per IP | | | |
| **Auth Limiter** | Login route only | 10 requests per 1 hour per | |
| IP | | | |

When rate limited, the API returns ``429 Too Many Requests``.

6. Data Models (Schemas)

6.1 User

```

```json
{
 "_id": "ObjectId",
 "full_name": "String (required, 2-50 chars)",
 "email": "String (required, unique, lowercase)",
 "phone_number": "String (optional, unique)",
 "profile_picture": "String (URL, optional)",
 "status": "String - 'active' | 'suspended' | 'unverified' (default:
'unverified')",
 "role": "String - 'STUDENT' | 'ADMIN' | 'INSTRUCTOR' (default:
'STUDENT')",
 "createdAt": "ISO Date",
 "updatedAt": "ISO Date"
}
```

```

> ⚠️ ``password_hash`` and ``refresh_token`` are ****never**** returned in any API response.

6.2 Category

```

```json
{
 "_id": "ObjectId",
 "name": "String (required, unique, 2-100 chars)",

```

```
 "description": "String (optional)",
 "createdAt": "ISO Date",
 "updatedAt": "ISO Date"
}
```
```

6.3 Course

```
```json
{
 "_id": "ObjectId",
 "title": "String (required, min 3 chars)",
 "description": "String (required, min 10 chars)",
 "price": "Number (required, >= 0, default: 0)",
 "access_type": "String - 'free' | 'paid' (default: 'paid')",
 "category": "ObjectId (ref: Category) - populated as { _id, name }",
 "created_by": "ObjectId (ref: User)",
 "is_active": "Boolean (default: true)",
 "isDeleted": "Boolean (default: false)",
 "deletedAt": "Date | null",
 "createdAt": "ISO Date",
 "updatedAt": "ISO Date"
}
```
```

6.4 Enrollment

```
```json
{
 "_id": "ObjectId",
 "user": "ObjectId (ref: User)",
 "course": "ObjectId (ref: Course)",
 "status": "String - 'ACTIVE' | 'EXPIRED' | 'REVOKED' (default:
'ACTIVE')",
 "enrolled_at": "ISO Date",
 "access_expires_at": "Date | null",
 "createdAt": "ISO Date",
 "updatedAt": "ISO Date"
}
```
```



```
> A user can only be enrolled in a specific course once (unique index:
`user + course`).
```

6.5 MockTest

```
```json
{
 "_id": "ObjectId",
 "title": "String (required, min 3 chars)",
 "description": "String (optional)",
 "course": "ObjectId (ref: Course, optional)",
 "category": "ObjectId (ref: Category, optional)",
 "difficulty": "String - 'easy' | 'medium' | 'hard' (default: 'medium')",
 "total_questions": "Number (required, >= 1)",
 "passing_marks": "Number (required, >= 1)",
 "negative_marking": "Boolean (default: false)",
 "negative_marks_per_wrong": "Number (default: 0)",
 "total_marks": "Number (required, >= 1)",
 "duration_minutes": "Number (required, >= 1)",
 "created_by": "ObjectId (ref: User)",
 "is_active": "Boolean (default: true)",
 "isDeleted": "Boolean (default: false)",
 "deletedAt": "Date | null",
 "questions": [
 {
 "_id": "ObjectId",
 "text": "String (required)",
 "marks": "Number (default: 1)",
 "explanation": "String (optional)",
 "options": [
 {
 "_id": "ObjectId",
 "text": "String (required)",
 "is_correct": "Boolean (required)"
 }
]
 }
],
 "createdAt": "ISO Date",

```

```

 "updatedAt": "ISO Date"
 }
 ...

> ⚠️ **Important:** For STUDENT users, the field `options.is_correct` is
stripped from the response (hidden). Only ADMIN users can see correct
answers.

```

### ### 6.6 TestAttempt

```

```json
{
  "_id": "ObjectId",
  "user": "ObjectId (ref: User)",
  "mock_test": "ObjectId (ref: MockTest)",
  "started_at": "ISO Date",
  "completed_at": "ISO Date | null",
  "status": "String - 'IN_PROGRESS' | 'COMPLETED' | 'ABANDONED' (default:
'IN_PROGRESS')",
  "answers": [
    {
      "question_id": "ObjectId",
      "question_text": "String",
      "selected_option_id": "ObjectId | null",
      "selected_option_text": "String | null",
      "is_correct": "Boolean (default: false)",
      "is_marked_for_review": "Boolean (default: false)",
      "answered_at": "ISO Date"
    }
  ],
  "score": "Number (default: 0)",
  "percentage": "Number (default: 0)",
  "rank": "Number | null",
  "feedback": "String | null",
  "createdAt": "ISO Date",
  "updatedAt": "ISO Date"
}
```

```

### ### 6.7 Payment

```

```json
{
  "_id": "ObjectId",
  "user": "ObjectId (ref: User)",
  "amount": "Number (required)",
  "currency": "String (default: 'INR')",
  "razorpay_order_id": "String (required, unique)",
  "razorpay_payment_id": "String | null",
  "razorpay_signature": "String | null",
  "status": "String - 'PENDING' | 'SUCCESS' | 'FAILED' | 'REFUNDED'
(default: 'PENDING')",
  "createdAt": "ISO Date",
  "updatedAt": "ISO Date"
}
```

```

### ### 6.8 Purchase

```

```json
{
  "_id": "ObjectId",
  "user": "ObjectId (ref: User)",
  "course": "ObjectId (ref: Course)",
  "payment": "ObjectId (ref: Payment)",
  "purchase_date": "ISO Date",
  "access_expires_at": "Date | null",
  "status": "String - 'ACTIVE' | 'EXPIRED' | 'REFUNDED' (default:
'ACTIVE')",
  "createdAt": "ISO Date",
  "updatedAt": "ISO Date"
}
```

```

### ### 6.9 Notification

```

```json
{
  "_id": "ObjectId",
  "user": "ObjectId (ref: User)",

```

```

    "title": "String (required)",
    "message": "String (required)",
    "type": "String - 'SYSTEM' | 'PURCHASE' | 'COURSE_UPDATE' |
'TEST_RESULT' (default: 'SYSTEM')",
    "is_read": "Boolean (default: false)",
    "createdAt": "ISO Date",
    "updatedAt": "ISO Date"
  }
  ...

```

>  Notifications ****auto-delete after 30 days**** (TTL index).

6.10 Resource

```

```json
{
 "_id": "ObjectId",
 "title": "String (required, min 3 chars)",
 "description": "String (optional)",
 "course": "ObjectId (ref: Course)",
 "file_url": "String (S3 key)",
 "resource_type": "String - 'pdf' | 'video' | 'notes' | 'assignment' |
'solution'",
 "created_by": "ObjectId (ref: User)",
 "is_active": "Boolean (default: true)",
 "isDeleted": "Boolean (default: false)",
 "deletedAt": "Date | null",
 "createdAt": "ISO Date",
 "updatedAt": "ISO Date"
}
...

```

### ### 6.11 Inquiry

```

```json
{
  "_id": "ObjectId",
  "student": "ObjectId (ref: User)",
  "subject": "String (required)",
  "message": "String (required)",

```

```
    "status": "String - 'OPEN' | 'IN_PROGRESS' | 'RESOLVED' | 'CLOSED'
(default: 'OPEN')",
    "admin_reply": "String | null",
    "replied_by": "ObjectId (ref: User) | null",
    "replied_at": "ISO Date | null",
    "createdAt": "ISO Date",
    "updatedAt": "ISO Date"
}
` ``
```

7. API Endpoints

7.1 Health Check

`GET /api/v1/healthcheck`

● ****Public****

****Description:**** Check if the server is running.

****Response:****

```
` ``json
{
  "statusCode": 200,
  "data": { "status": "ok" },
  "message": "OK",
  "success": true
}
` ``
```

7.2 Auth Routes

Base Path: `/api/v1/auth`

`POST /api/v1/auth/register`

● ****Public****

****Description:**** Register a new user account.

****Request Body (JSON):****

| Field | Type | Required | Validation |
|-------|-------|----------|------------|
| ----- | ----- | ----- | |

| | | | |
|---------------------------|--------|---|------------------------------------|
| --- | | | |
| `full_name` | String | ✓ | Min 2 chars, max 50 chars |
| | | | |
| `email` | String | ✓ | Must be a valid email |
| | | | |
| `password` | String | ✓ | Min 8 chars, must contain at least |
| one letter and one number | | | |
| `phone_number` | String | ✗ | Optional |
| | | | |

****Example Request:****

```
```json
{
 "full_name": "Rahul Sharma",
 "email": "rahul@example.com",
 "password": "SecurePass123",
 "phone_number": "9876543210"
}
```
```

****Success Response (201):****

```
```json
{
 "statusCode": 201,
 "data": {
 "_id": "665a...",
 }
}
```

```
 "full_name": "Rahul Sharma",
 "email": "rahul@example.com",
 "phone_number": "9876543210",
 "status": "unverified",
 "role": "STUDENT",
 "createdAt": "2026-06-12T...",
 "updatedAt": "2026-06-12T..."
 },
 "message": "User registered successfully",
 "success": true
}
...

```

#### **\*\*Error Responses:\*\***

```
- `409` - User with this email already exists
- `400` - Validation error

```

#### **#### `POST /api/v1/auth/login`**

 **\*\*Public\*\*** (rate limited: 10 attempts/hour)

**\*\*Description:\*\*** Login with email and password. Returns tokens in both cookies and response body.

#### **\*\*Request Body (JSON):\*\***

| Field      | Type   | Required | Validation            |
|------------|--------|----------|-----------------------|
| -----      | -----  | -----    | -----                 |
| `email`    | String | ✓        | Must be a valid email |
| `password` | String | ✓        | Must not be empty     |

#### **\*\*Example Request:\*\***

```
```json
{
  "email": "rahul@example.com",
  "password": "SecurePass123"
}
...

```

****Success Response (200):****

Sets cookies: `accessToken`, `refreshToken` (httpOnly, secure in production, sameSite: strict)

```
```json
{
 "statusCode": 200,
 "data": {
 "user": {
 "_id": "665a...",
 "full_name": "Rahul Sharma",
 "email": "rahul@example.com",
 "role": "STUDENT",
 "status": "unverified"
 },
 "accessToken": "eyJhbGciOiJIUzI1NiIs...",
 "refreshToken": "eyJhbGciOiJIUzI1NiIs..."
 },
 "message": "User logged in successfully",
 "success": true
}
```
```

****Error Responses:****

- `401` - Invalid credentials
- `403` - Account is temporarily locked (after 5 failed attempts, locked for 15 minutes)
- `429` - Too many login attempts

>  ****Account Locking:**** After ****5 consecutive failed login attempts****, the account is locked for ****15 minutes****.

`POST /api/v1/auth/logout`

 ****Auth Required****

****Description:**** Logout current user. Clears refresh token from DB and cookies.

****Request Body:**** None

****Headers:****

```

Authorization: Bearer <accessToken>

```

****Success Response (200):****

```json

```
{
 "statusCode": 200,
 "data": {},
 "message": "User logged out successfully",
 "success": true
}
```

```

`POST /api/v1/auth/refresh-token`

 ****Public**** (uses refresh token, not access token)

****Description:**** Get a new pair of access + refresh tokens using the existing refresh token. Implements ****token rotation**** (old refresh token is invalidated).

****Request Body (JSON):****

| Field | Type | Required | Notes |
|-------|------|----------|-------|
|-------|------|----------|-------|

| | | | |
|--|--|--|--|
| | | | |
|--|--|--|--|

| | | | |
|-------|-------|-------|--|
| ----- | ----- | ----- | |
|-------|-------|-------|--|

| | | | |
|-------|--|--|--|
| ----- | | | |
|-------|--|--|--|

| | | | |
|----------------|--------|---|--|
| `refreshToken` | String | ✗ | Can also be sent via `refreshToken` cookie instead |
|----------------|--------|---|--|

****Success Response (200):****

Sets new cookies: `accessToken`, `refreshToken`

```
```json
{
 "statusCode": 200,
 "data": {
 "accessToken": "eyJhbGciOiJIUzI1NiIs... ",
 "refreshToken": "eyJhbGciOiJIUzI1NiIs..."
 },
 "message": "Access token refreshed",
 "success": true
}
```
```

****Error Responses:****

- `401` - Invalid/expired refresh token or token reuse detected

>  ****Security:**** If a refresh token is reused (e.g., stolen and used by attacker), the system ****revokes all tokens**** for that user, requiring re-login.

`POST /api/v1/auth/change-password`

 ****Auth Required****

****Description:**** Change the password of the currently logged-in user. Invalidates all refresh tokens after change.

****Request Body (JSON):****

| Field | Type | Required | Validation |
|---------------|--------|---|-------------------|
| ----- | ----- | ----- | ----- |
| `oldPassword` | String |  | Must not be empty |
| `newPassword` | String |  | Min 8 characters |

****Example Request:****

```
```json
```

```
{
 "oldPassword": "OldPass123",
 "newPassword": "NewSecure456"
}
```
```

****Success Response (200):****

```
```json
{
 "statusCode": 200,
 "data": {},
 "message": "Password changed successfully",
 "success": true
}
```
```

****Error Responses:****

- `400` - Invalid old password

`POST /api/v1/auth/forgot-password`

****Public****

****Description:**** Request a password reset. Generates a reset token (valid for 10 minutes). Currently mocks email sending – the reset token URL is logged on the server.

****Request Body (JSON):****

| Field | Type | Required | Validation |
|---------|--------|---|-----------------------|
| ----- | ----- | ----- | ----- |
| `email` | String |  | Must be a valid email |

****Success Response (200):****

```
```json
{
 "statusCode": 200,
 "data": {},

```

```
 "message": "Password reset token generated and sent to email",
 "success": true
}
```
```

****Error Responses:****

- `404` - User with this email does not exist

`POST /api/v1/auth/reset-password/:token`

● ****Public****

****Description:**** Reset password using the token received via email.
Invalidates all refresh tokens after reset.

****URL Params:****

| Param | Type | Description |
|---------|--------|------------------------------|
| ----- | ----- | ----- |
| `token` | String | The reset token from the URL |

****Request Body (JSON):****

| Field | Type | Required | Validation |
|------------|--------|----------|------------------|
| ----- | ----- | ----- | ----- |
| `password` | String | ✓ | Min 8 characters |

****Success Response (200):****

```
```json
{
 "statusCode": 200,
 "data": {},
 "message": "Password reset successfully",
 "success": true
}
```
```

****Error Responses:****

```
- `400` - Invalid or expired reset token
```

```
---
```

7.3 User Routes

Base Path: `/api/v1/users`

> All routes in this section require authentication.

```
---
```

`GET /api/v1/users/me`



****Auth Required****

****Description:**** Get the current logged-in user's profile.

****Response (200):****

```json

```
{
 "statusCode": 200,
 "data": {
 "_id": "665a...",
 "full_name": "Rahul Sharma",
 "email": "rahul@example.com",
 "phone_number": "9876543210",
 "profile_picture": null,
 "status": "active",
 "role": "STUDENT",
 "createdAt": "...",
 "updatedAt": "..."
 },
 "message": "User profile fetched successfully",
 "success": true
}
```

```

```
---
```

`PATCH /api/v1/users/update-account`

 ****Auth Required****

****Description:**** Update the current user's profile information.

****Request Body (JSON):****

| Field | Type | Required | Validation |
|----------------|--------|----------|-------------------------------------|
| ----- | ----- | ----- | ----- |
| `full_name` | String | ✗ | Min 2 chars, max 50 chars (if sent) |
| `phone_number` | String | ✗ | Optional |

> At least one field must be provided.

****Success Response (200):**** Returns updated user object.

`PATCH /api/v1/users/avatar`

 ****Auth Required****

****Description:**** Update the current user's profile picture.

****Request Body (JSON):****

| Field | Type | Required | Validation |
|-------------------|--------|----------|---------------|
| ----- | ----- | ----- | ----- |
| `profile_picture` | String | ✓ | Must be a URL |

****Example Request:****

```
```json
{
 "profile_picture": "https://cloudinary.com/images/avatar123.jpg"
}
```

\*\*\*  
\*\*Success Response (200):\*\* Returns updated user object.

---

#### `GET /api/v1/users`

● \*\*Admin Only\*\*

\*\*Description:\*\* Get all users in the system.

\*\*Response (200):\*\* Returns array of user objects.

---

#### `GET /api/v1/users/:id`

● \*\*Admin Only\*\*

\*\*Description:\*\* Get a specific user by ID.

\*\*URL Params:\*\*

|       |        |                  |  |
|-------|--------|------------------|--|
| Param | Type   | Description      |  |
| ----- | -----  | -----            |  |
| `id`  | String | MongoDB ObjectId |  |

\*\*Success Response (200):\*\* Returns single user object.

\*\*Error:\*\* `404` – User not found.

---

#### `PATCH /api/v1/users/:id/status`

● \*\*Admin Only\*\*

\*\*Description:\*\* Update a user's account status.

\*\*URL Params:\*\*

| Param | Type   | Description      |  |
|-------|--------|------------------|--|
| ----- | -----  | -----            |  |
| `id`  | String | MongoDB ObjectId |  |

#### **\*\*Request Body (JSON):\*\***

| Field    | Type   | Required                                                                          | Validation                                          |
|----------|--------|-----------------------------------------------------------------------------------|-----------------------------------------------------|
| -----    | -----  | -----                                                                             |                                                     |
| -----    |        |                                                                                   |                                                     |
| `status` | String |  | Must be one of: `active`, `suspended`, `unverified` |

**\*\*Success Response (200):\*\*** Returns updated user object.

---

#### **#### `DELETE /api/v1/users/:id`**

 **\*\*Admin Only\*\***

**\*\*Description:\*\*** Permanently delete a user.

#### **\*\*URL Params:\*\***

| Param | Type   | Description      |  |
|-------|--------|------------------|--|
| ----- | -----  | -----            |  |
| `id`  | String | MongoDB ObjectId |  |

#### **\*\*Success Response (200):\*\***

```
```json
{
  "statusCode": 200,
  "data": {},
  "message": "User deleted successfully",
  "success": true
}
```
```



----

### ### 7.4 Category Routes

Base Path: `/api/v1/categories`

----

#### #### `GET /api/v1/categories`

● **\*\*Public\*\***

**\*\*Description:\*\*** Get all categories.

**\*\*Response (200):\*\***

```json

```
{
  "statusCode": 200,
  "data": [
    {
      "_id": "665a...",
      "name": "UPSC",
      "description": "UPSC exam preparation",
      "createdAt": "...",
      "updatedAt": "..."
    }
  ],
  "message": "Categories fetched successfully",
  "success": true
}
```

```

----

#### #### `GET /api/v1/categories/:id`

● **\*\*Public\*\***

**\*\*Description:\*\*** Get a single category by ID.

**\*\*Error:\*\*** `404` – Category not found.

---

#### `POST /api/v1/categories`

● **\*\*Admin Only\*\***

**\*\*Description:\*\*** Create a new category.

**\*\*Request Body (JSON):\*\***

| Field         | Type   | Required | Validation                         |
|---------------|--------|----------|------------------------------------|
| -----         | -----  | -----    | -----                              |
| `name`        | String | ✓        | Min 2 chars, max 100 chars, unique |
| `description` | String | ✗        | Optional                           |

**\*\*Success Response (201):\*\*** Returns created category object.

**\*\*Error:\*\*** `400` – Category already exists.

---

#### `PUT /api/v1/categories/:id`

● **\*\*Admin Only\*\***

**\*\*Description:\*\*** Update a category.

**\*\*Request Body (JSON):\*\***

| Field         | Type   | Required | Validation                 |
|---------------|--------|----------|----------------------------|
| -----         | -----  | -----    | -----                      |
| `name`        | String | ✗        | Min 2 chars, max 100 chars |
| `description` | String | ✗        | Optional                   |

**\*\*Success Response (200):\*\*** Returns updated category object.

---

#### `DELETE /api/v1/categories/:id`

🔴 **\*\*Admin Only\*\***

**\*\*Description:\*\*** Delete a category.

**\*\*Success Response (200):\*\*** Returns empty `data`.

---

### ### 7.5 Course Routes

Base Path: `/api/v1/courses`

> All routes require authentication.

---

#### `GET /api/v1/courses`

🔒 **\*\*Auth Required\*\***

**\*\*Description:\*\*** Get all courses. Students only see active (`is\_active: true`) courses. Admins see all non-deleted courses.

**\*\*Response (200):\*\***

```json

```
{
  "statusCode": 200,
  "data": [
    {
      "_id": "665a...",
      "title": "UPSC Prelims 2026",
      "description": "Comprehensive course...",
      "price": 999,
      "access_type": "paid",
      "category": { "_id": "...", "name": "UPSC" },
      "created_by": "...",
      "is_active": true,
      "isDeleted": false,
      "createdAt": "...",
```

```
      "updatedAt": "..."  
    }  
  ],  
  "message": "Courses fetched successfully",  
  "success": true  
}  
...  
---
```

`GET /api/v1/courses/:id`

 ****Auth Required****

****Description:**** Get a single course by ID. Students can only see active courses.

****Error:**** `404` – Course not found.

`GET /api/v1/courses/my/enrolled`

 ****Auth Required****

****Description:**** Get all courses the current student is enrolled in.

****Response (200):**** Returns array of course objects (populated with category).

`POST /api/v1/courses/:id/enroll`

 ****Auth Required**** (Students only for free courses)

****Description:**** Enroll in a free course. Paid courses must be purchased through the payment flow.

****Request Body:**** None

****Success Response (201):****

```
```json
{
 "statusCode": 201,
 "data": {
 "_id": "...",
 "user": "...",
 "course": "...",
 "status": "ACTIVE"
 },
 "message": "Successfully enrolled in course",
 "success": true
}
```
```

****Error Responses:****

- `400` - Paid courses require purchase
- `400` - Already enrolled
- `404` - Course not found or inactive

`POST /api/v1/courses`

****Admin Only****

****Description:**** Create a new course.

****Request Body (JSON):****

| Field | Type | Required | Validation |
|---------------|--------|---|--------------|
| ----- | ----- | ----- | ----- |
| `title` | String |  | Min 3 chars |
| `description` | String |  | Min 10 chars |

| | | | |
|---------------|--------|---|---|
| `price` | Number | ✗ | >= 0, default: 0 |
| | | | |
| `access_type` | String | ✗ | `"free"` or `"paid"`, default: `"paid"` |
| | | | |
| `category` | String | ✓ | Valid 24-char hex MongoDB ObjectId string |

****Success Response (201):**** Returns created course object.

`PUT /api/v1/courses/:id`

● ****Admin Only****

****Description:**** Update a course. Accepts any subset of the course fields.

****Request Body (JSON):**** Same fields as create, all optional.

****Success Response (200):**** Returns updated course object.

`DELETE /api/v1/courses/:id`

● ****Admin Only****

****Description:**** Soft-delete a course (sets `isDeleted: true`).

`PATCH /api/v1/courses/:id/publish`

● ****Admin Only****

****Description:**** Set course `is\_active` to `true`.

****Request Body:**** None

****Success Response (200):**** Returns updated course object.

`PATCH /api/v1/courses/:id/unpublish`

 ****Admin Only****

****Description:**** Set course `is\_active` to `false`.

****Request Body:**** None

****Success Response (200):**** Returns updated course object.

7.6 Resource Routes

Base Path: `/api/v1/resources`

> All routes require authentication.

`GET /api/v1/resources/course/:courseId`

 ****Auth Required**** (must be enrolled in the course OR be admin)

****Description:**** Get all resources for a specific course.

****URL Params:****

| Param | Type | Description |
|------------|--------|------------------|
| ----- | ----- | ----- |
| `courseId` | String | MongoDB ObjectId |

****Response (200):**** Returns array of resource objects.

****Errors:****

- `403` - Active enrollment required
- `404` - Course not found

`GET /api/v1/resources/:id/download`



****Auth Required**** (must be enrolled OR be admin)

****Description:**** Get a ****signed S3 download URL**** for a resource file. The signed URL is ****time-limited****.

****URL Params:****

| Param | Type | Description |
|-------|--------|-------------------|
| ----- | ----- | ----- |
| `id` | String | Resource Objectid |

****Success Response (200):****

```json

```
{
 "statusCode": 200,
 "data": {
 "downloadUrl":
"https://s3.amazonaws.com/bucket/...?X-Amz-Signature=..."
 },
 "message": "Signed URL generated successfully",
 "success": true
}
```

```

>  The `downloadUrl` is a ****pre-signed S3 URL****. Open it in a browser or use it in `` / `window.open()` for download. It expires after a limited time.

`POST /api/v1/resources`



****Admin Only****

****Description:**** Upload a new resource file to a course.

****Content-Type:**** `multipart/form-data`

| Field | Type | Location | Required | Validation |
|-----------------|--------|-----------|----------|---|
| ----- | ----- | ----- | ----- | |
| ----- | | | | |
| `file` | File | Form Data | ✓ | Max 50 MB |
| `title` | String | Form Data | ✓ | Min 3 chars |
| `description` | String | Form Data | ✗ | Optional |
| `course` | String | Form Data | ✓ | Valid MongoDB ObjectId |
| `resource_type` | String | Form Data | ✓ | `"pdf"`, `"video"`,
`"notes"`, `"assignment"`, or `"solution"` |

> ⚠ This is the ****only endpoint that uses `multipart/form-data`**** instead of JSON. Use `FormData` in the frontend.

****Success Response (201):**** Returns created resource object.

`DELETE /api/v1/resources/:id`

● ****Admin Only****

****Description:**** Delete a resource (removes from S3 and database permanently).

****Success Response (200):**** Returns empty `data`.

7.7 Mock Test Routes

Base Path: `/api/v1/mock-tests`

```

> All routes require authentication.

---

#### `GET /api/v1/mock-tests`

🔒 **Auth Required**

**Description:** Get all mock tests. Students only see active tests.
**Correct answers (`is_correct`) are hidden** from students.

**Response (200):** Returns array of mock test objects.

---

#### `GET /api/v1/mock-tests/:id`

🔒 **Auth Required**

**Description:** Get a single mock test by ID. **Correct answers are hidden** for non-admin users.

**Error:** `404` - Mock Test not found.

---

#### `POST /api/v1/mock-tests`

🔴 **Admin Only**

**Description:** Create a new mock test.

**Request Body (JSON):**

| Field   | Type   | Required | Validation / Default |
|---------|--------|----------|----------------------|
| -----   | -----  | -----    |                      |
| -----   | -----  | -----    |                      |
| `title` | String | ✓        | Min 3 chars          |


```

| | | | |
|----------------------------|---------|---|---|
| `description` | String | ❌ | Optional |
| | | | |
| `course` | String | ❌ | Valid MongoDB ObjectId |
| | | | |
| `category` | String | ❌ | Valid MongoDB ObjectId |
| | | | |
| `difficulty` | String | ❌ | `"easy"`, `"medium"`,
`"hard"` (default: `"medium"`) |
| `total_questions` | Number | ✅ | >= 1 |
| | | | |
| `passing_marks` | Number | ✅ | >= 1 |
| | | | |
| `negative_marking` | Boolean | ❌ | Default: `false` |
| | | | |
| `negative_marks_per_wrong` | Number | ❌ | >= 0, default: 0 |
| | | | |
| `total_marks` | Number | ✅ | >= 1 |
| | | | |
| `duration_minutes` | Number | ✅ | >= 1 |
| | | | |

****Success Response (201):**** Returns created mock test object.

`PUT /api/v1/mock-tests/:id`

🔴 ****Admin Only****

****Description:**** Update a mock test. All fields are optional (partial update).

`DELETE /api/v1/mock-tests/:id`

🔴 ****Admin Only****

****Description:**** Soft-delete a mock test.

`PATCH /api/v1/mock-tests/:id/publish`

● ****Admin Only****

****Description:**** Publish a mock test (`is\_active: true`).

****Request Body:**** None

`PATCH /api/v1/mock-tests/:id/unpublish`

● ****Admin Only****

****Description:**** Unpublish a mock test (`is\_active: false`).

****Request Body:**** None

`POST /api/v1/mock-tests/:id/questions`

● ****Admin Only****

****Description:**** Add a single question to a mock test.

****Request Body (JSON):****

| Field | Type | Required | Validation |
|---------------|--------|----------|------------------------|
| ----- | ----- | ----- | ----- |
| `text` | String | ✓ | Min 1 char |
| `marks` | Number | ✗ | >= 1, default: 1 |
| `explanation` | String | ✗ | Optional |
| `options` | Array | ✓ | Min 2 options required |

****Each Option:****

| Field | Type | Required | Description |
|-------|------|----------|-------------|
|-------|------|----------|-------------|

| Field | Type | Required | Validation |
|--------------|---------|----------|----------------------------|
| `text` | String | ✓ | Option text, min 1 char |
| `is_correct` | Boolean | ✓ | Whether this is the answer |

Example Request:

```
```json
{
 "text": "What is the capital of India?",
 "marks": 2,
 "explanation": "New Delhi is the capital of India",
 "options": [
 { "text": "Mumbai", "is_correct": false },
 { "text": "New Delhi", "is_correct": true },
 { "text": "Kolkata", "is_correct": false },
 { "text": "Chennai", "is_correct": false }
]
}
```

**Success Response (201):** Returns the created question object.

---

#### `POST /api/v1/mock-tests/:id/questions/bulk`

 **Admin Only**

**Description:** Add multiple questions at once.

#### **Request Body (JSON):**

| Field       | Type  | Required | Validation                        |
|-------------|-------|----------|-----------------------------------|
| -----       | ----- | -----    | -----                             |
| `questions` | Array | ✓        | Array of question objects (min 1) |

Each question follows the same format as the single question endpoint above.

**Success Response (201):** Returns the updated mock test object with all questions.

---

#### `PUT /api/v1/mock-tests/:id/questions/:questionId`

 **\*\*Admin Only\*\***

**\*\*Description:\*\*** Update a specific question.

**\*\*URL Params:\*\***

| Param        | Type   | Description        |
|--------------|--------|--------------------|
| -----        | -----  | -----              |
| `id`         | String | Mock Test ObjectId |
| `questionId` | String | Question ObjectId  |

**\*\*Request Body:\*\*** Same format as add question.

---

#### `DELETE /api/v1/mock-tests/:id/questions/:questionId`

 **\*\*Admin Only\*\***

**\*\*Description:\*\*** Delete a specific question from a mock test.

---

## ### 7.8 Test Attempt Routes

Base Path: `/api/v1/attempts`

> All routes require authentication.

---

#### `POST /api/v1/attempts/start`

 **\*\*Auth Required\*\***

**\*\*Description:\*\*** Start a new test attempt. If an ``IN_PROGRESS`` attempt already exists for the same test, it will be returned instead of creating a new one. A **\*\*BullMQ auto-submit job\*\*** is scheduled to automatically submit the test when the duration expires.

**\*\*Request Body (JSON):\*\***

| Field                       | Type   | Required                                                                          | Validation             |
|-----------------------------|--------|-----------------------------------------------------------------------------------|------------------------|
| -----                       | -----  | -----                                                                             | -----                  |
| <code>`mock_test_id`</code> | String |  | Valid MongoDB ObjectId |

**\*\*Example Request:\*\***

```
```json
{
  "mock_test_id": "665a1234abcd5678ef901234"
}
```
```

**\*\*Success Response (201):\*\***

```
```json
{
  "statusCode": 201,
  "data": {
    "_id": "...",
    "user": "...",
    "mock_test": "665a1234abcd5678ef901234",
    "started_at": "2026-06-12T...",
    "status": "IN_PROGRESS",
    "answers": [],
    "score": 0,
    "percentage": 0
  },
  "message": "Test started successfully",
  "success": true
}
```
```

**\*\*Error Responses:\*\***

- ``400`` - Maximum attempt limit reached (max 3 attempts per test)
- ``404`` - Mock test not found or inactive

```
> ⚠️ **Max 3 attempts** allowed per user per mock test.
```

```

```

```
`PUT /api/v1/attempts/:attemptId/answer`
```

```
🔒 **Auth Required**
```

**\*\*Description:\*\*** Save or update an answer for a question in an active test attempt. Can also mark a question for review.

**\*\*URL Params:\*\***

| Param       | Type   | Description          |
|-------------|--------|----------------------|
| -----       | -----  | -----                |
| `attemptId` | String | TestAttempt ObjectId |

**\*\*Request Body (JSON):\*\***

| Field                  | Type    | Required | Validation                                     |
|------------------------|---------|----------|------------------------------------------------|
| -----                  | -----   | -----    | -----                                          |
| `question_id`          | String  | ✓        | Valid MongoDB ObjectId                         |
| `selected_option_id`   | String  | ✗        | Valid MongoDB ObjectId or `null` (to deselect) |
| `is_marked_for_review` | Boolean | ✗        | Default: `false`                               |

**\*\*Example Request:\*\***

```
```json
{
  "question_id": "665a...",
  "selected_option_id": "665b...",
  "is_marked_for_review": false
}
```
```



**\*\*Success Response (200):\*\*** Returns the updated ``answers`` array.

**\*\*Error Responses:\*\***

- ``404`` - Active test attempt not found
- ``404`` - Question not found in this test
- ``404`` - Option not found

---

#### ``POST /api/v1/attempts/:attemptId/submit``

 **\*\*Auth Required\*\***

**\*\*Description:\*\*** Manually submit the test. Changes status from ``IN_PROGRESS`` to ``COMPLETED``.

**\*\*URL Params:\*\***

| Param                    | Type   | Description          |
|--------------------------|--------|----------------------|
| -----                    | -----  | -----                |
| <code>`attemptId`</code> | String | TestAttempt ObjectId |

**\*\*Request Body:\*\*** None

**\*\*Success Response (200):\*\***

```
```json
{
  "statusCode": 200,
  "data": {
    "_id": "...",
    "status": "COMPLETED",
    "completed_at": "2026-06-12T...",
    "answers": [...],
    "score": 0,
    "percentage": 0
  },
  "message": "Test submitted successfully",
  "success": true
}
```
```

> ⚠ Tests are also **auto-submitted** when the duration expires (via background BullMQ job).

---

#### `POST /api/v1/attempts/:attemptId/evaluate`

🔒 **Auth Required**

**Description:** Evaluate/grade a completed test attempt. Calculates score based on correct answers, applies negative marking if enabled.

**URL Params:**

| Param       | Type   | Description          |
|-------------|--------|----------------------|
| -----       | -----  | -----                |
| `attemptId` | String | TestAttempt ObjectId |

**Request Body:** None

**Success Response (200):**

```
```json
{
  "statusCode": 200,
  "data": {
    "_id": "...",
    "status": "COMPLETED",
    "answers": [
      {
        "question_id": "...",
        "question_text": "What is the capital of India?",
        "selected_option_id": "...",
        "selected_option_text": "New Delhi",
        "is_correct": true,
        "is_marked_for_review": false,
        "answered_at": "..."
      }
    ],
    "score": 18,
  }
}
```

```
    "percentage": 90
  },
  "message": "Test evaluated successfully",
  "success": true
}
...

```

****Error Responses:****

- `400` - Test is not completed yet
- `404` - Attempt not found

> ⚠️ **Scoring Logic:**

- > - Correct answer: `+question.marks`
- > - Wrong answer (if `negative\_marking: true`):
`-mockTest.negative\_marks\_per\_wrong`
- > - Unanswered: `0`
- > - Score is clamped to minimum `0` (never negative)

`GET /api/v1/attempts/:attemptId`

🔒 **Auth Required**

****Description:**** Get a specific test attempt by ID.

****Success Response (200):**** Returns the full test attempt object.

7.9 Payment Routes (Razorpay)

Base Path: `/api/v1/payments`

> All routes require authentication.

`POST /api/v1/payments/create-order`

****Auth Required****

****Description:**** Create a Razorpay order for purchasing a course or item.

****Request Body (JSON):****

| Field | Type | Required | Description |
|-------------|--------|----------|--|
| ----- | ----- | ----- | |
| ----- | | | |
| `item_id` | String | ✓ | MongoDB ObjectId of the item being purchased |
| `item_type` | String | ✓ | Type of item (e.g., `"Course"`) |
| `amount` | Number | ✓ | Amount in **INR** (rupees, NOT paise) |

****Example Request:****

```
```json
{
 "item_id": "665a...",
 "item_type": "Course",
 "amount": 999
}
```

### **\*\*Success Response (200):\*\***

```
```json
{
  "statusCode": 200,
  "data": {
    "id": "order_NxP...",
    "entity": "order",
    "amount": 99900,
    "amount_paid": 0,
    "amount_due": 99900,
    "currency": "INR",
    "receipt": "receipt_order_1718...",
    "status": "created"
  },
}
```

```

    "message": "Order created",
    "success": true
  }
  ...

> ⚠ The `data.id` (Razorpay Order ID) is needed for opening the Razorpay
checkout modal.
> ⚠ The `data.amount` in the response is in **paise** (amount × 100).

---

#### `POST /api/v1/payments/verify`

🔒 **Auth Required**

**Description:** Verify Razorpay payment after the user completes payment
in the Razorpay modal. On success, automatically creates a **Purchase**
record and auto-enrolls the user if the item is a Course.

**Request Body (JSON):**

| Field | Type | Required | Description |
| --- | --- | --- | --- |
| `razorpay_order_id` | String | ✓ | The order ID from
create-order response |
| `razorpay_payment_id` | String | ✓ | From Razorpay success
callback |
| `razorpay_signature` | String | ✓ | From Razorpay success
callback |

**Example Request:**

```json
{
 "razorpay_order_id": "order_NxP...",
 "razorpay_payment_id": "pay_NxQ...",
 "razorpay_signature": "a1b2c3d4e5..."
}
...

```

### **\*\*Success Response (200):\*\***

```
```json
{
  "statusCode": 200,
  "data": {
    "_id": "...",
    "user": "...",
    "order_id": "order_NxP...",
    "amount": 999,
    "currency": "INR",
    "payment_status": "SUCCESS",
    "payment_id": "pay_NxQ..."
  },
  "message": "Payment verified successfully",
  "success": true
}
```
```

### **\*\*Error Responses:\*\***

- `400` - Payment verification failed (signature mismatch)
- `404` - Order not found

### **> \*\*Frontend Integration Flow:\*\***

- > 1. Call `POST /payments/create-order` to get a Razorpay order ID
- > 2. Open Razorpay checkout modal with the order ID
- > 3. On success callback, send the 3 Razorpay values to `POST /payments/verify`
- > 4. On success, the user is auto-enrolled in the course

---

### **### 7.10 Notification Routes**

Base Path: `/api/v1/notifications`

- > All routes require authentication.

---

```
`GET /api/v1/notifications`
```

 **\*\*Auth Required\*\***

**\*\*Description:\*\*** Get all notifications for the current user, sorted by most recent first.

**\*\*Response (200):\*\***

```
```json
{
  "statusCode": 200,
  "data": [
    {
      "_id": "...",
      "user": "...",
      "title": "Course Enrolled",
      "message": "You've been enrolled in UPSC Prelims 2026",
      "type": "PURCHASE",
      "is_read": false,
      "createdAt": "..."
    }
  ],
  "message": "Notifications fetched",
  "success": true
}
```
```

---

```
`PATCH /api/v1/notifications/:id/read`
```

 **\*\*Auth Required\*\***

**\*\*Description:\*\*** Mark a notification as read.

**\*\*URL Params:\*\***

| Param | Type   | Description           |
|-------|--------|-----------------------|
| ----- | -----  | -----                 |
| `id`  | String | Notification ObjectId |

**\*\*Request Body:\*\*** None

**\*\*Success Response (200):\*\*** Returns the updated notification with  
`is\_read: true`.

---

### ### 7.11 Dashboard Routes

Base Path: `/api/v1/dashboard`

> All routes require authentication.

---

#### `GET /api/v1/dashboard/student`



**\*\*Auth Required\*\***

**\*\*Description:\*\*** Get the student's dashboard statistics.

**\*\*Response (200):\*\***

```json

```
{
  "statusCode": 200,
  "data": {
    "totalAttempts": 15,
    "avgScore": "78.50",
    "recentActivity": [
      {
        "_id": "...",
        "mock_test": "...",
        "score": 85,
        "percentage": 90,
        "status": "COMPLETED",
        "completed_at": "..."
      }
    ]
  }
},
```



```
    "message": "Student dashboard fetched",
    "success": true
}
```

> `avgScore` is returned as a string (formatted to 2 decimal places).
> `recentActivity` contains the last 5 completed test attempts.
```

---

#### `GET /api/v1/dashboard/admin``

 **Admin Only**

**Description:** Get the admin dashboard statistics.

**Response (200):**

```
```json
{
  "statusCode": 200,
  "data": {
    "totalStudents": 250,
    "totalCourses": 12,
    "totalTests": 45,
    "revenue": 125000
  },
  "message": "Admin dashboard fetched",
  "success": true
}
```
```

> `revenue` is the total amount (in INR) from all successful payments.

---

### ### 7.12 Leaderboard Routes

Base Path: `/api/v1/leaderboard``

> All routes require authentication.

---

#### `GET /api/v1/leaderboard/:testId`

 **\*\*Auth Required\*\***

**\*\*Description:\*\*** Get the top 10 leaderboard for a specific mock test. Sorted by highest score, then earliest completion. Results are **\*\*cached in Redis for 5 minutes\*\***.

**\*\*URL Params:\*\***

| Param    | Type   | Description       |
|----------|--------|-------------------|
| -----    | -----  | -----             |
| `testId` | String | MockTest ObjectId |

**\*\*Response (200):\*\***

```
```json
{
  "statusCode": 200,
  "data": [
    {
      "_id": "...",
      "user": { "_id": "...", "name": "...", "email": "..." },
      "mock_test": "...",
      "score": 95,
      "percentage": 100,
      "completed_at": "..."
    }
  ],
  "message": "Leaderboard fetched",
  "success": true
}
```
```

---

### 7.13 Inquiry Routes

Base Path: ``/api/v1/inquiries``

> All routes require authentication.

---

#### ``POST /api/v1/inquiries``

 **\*\*Auth Required\*\*** (Students)

**\*\*Description:\*\*** Submit a new inquiry/support ticket.

**\*\*Request Body (JSON):\*\***

| Field                  | Type   | Required | Description            |
|------------------------|--------|----------|------------------------|
| -----                  | -----  | -----    | -----                  |
| <code>`subject`</code> | String | ✓        | Subject of the inquiry |
| <code>`message`</code> | String | ✓        | Detailed message       |

**\*\*Example Request:\*\***

```
```json
{
  "subject": "Unable to access course materials",
  "message": "I purchased the UPSC course but cannot see the study
materials."
}
```
```

**\*\*Success Response (201):\*\***

```
```json
{
  "statusCode": 201,
  "data": {
    "_id": "...",
    "student": "...",
    "subject": "Unable to access course materials",
    "message": "I purchased the UPSC course...",
    "status": "OPEN",
    "createdAt": "..."
  },

```

```
    "message": "Inquiry submitted",
    "success": true
  }
  ...

```

```
#### `GET /api/v1/inquiries/my`
```

 ****Auth Required**** (Students)

****Description:**** Get all inquiries submitted by the current user.

****Response (200):**** Returns array of inquiry objects sorted by most recent.

```
#### `GET /api/v1/inquiries`
```

 ****Admin Only****

****Description:**** Get all inquiries from all students. Populated with student name and email.

****Response (200):****

```json

```
{
 "statusCode": 200,
 "data": [
 {
 "_id": "...",
 "student": { "_id": "...", "name": "...", "email": "..." },
 "subject": "...",
 "message": "...",
 "status": "OPEN",
 "createdAt": "..."
 }
],
 "message": "All inquiries",

```

```

 "success": true
 }
}
...

`PATCH /api/v1/inquiries/:id/reply`

🔴 **Admin Only**

Description: Reply to a student inquiry. Sets status to `RESOLVED`.

URL Params:

| Param | Type | Description |
|-------|--------|------------------|
| ----- | ----- | ----- |
| `id` | String | Inquiry ObjectId |

Request Body (JSON):

| Field | Type | Required | Description |
|---------|--------|----------|---------------|
| ----- | ----- | ----- | ----- |
| `reply` | String | ✓ | Admin's reply |

Example Request:

```

```json
{
 "reply": "We've fixed the issue. Please try accessing the materials
again."
}
...

```

**Success Response (200):**

```

```json
{
  "statusCode": 200,
  "data": {
    "_id": "...",
    "status": "RESOLVED",
    "admin_reply": "We've fixed the issue...",

```


```

```

 "replied_by": "...",
 "replied_at": "2026-06-12T..."
 },
 "message": "Inquiry replied",
 "success": true
}
...

```

---

## ## 8. Enum Values Reference

| Model        | Field           | Valid Values                                            |
|--------------|-----------------|---------------------------------------------------------|
| -----        | -----           | -----                                                   |
| User         | `role`          | `STUDENT`, `ADMIN`,<br>`INSTRUCTOR`                     |
| User         | `status`        | `active`, `suspended`,<br>`unverified`                  |
| Course       | `access_type`   | `free`, `paid`                                          |
| MockTest     | `difficulty`    | `easy`, `medium`, `hard`                                |
| Resource     | `resource_type` | `pdf`, `video`, `notes`,<br>`assignment`, `solution`    |
| Enrollment   | `status`        | `ACTIVE`, `EXPIRED`,<br>`REVOKED`                       |
| TestAttempt  | `status`        | `IN_PROGRESS`, `COMPLETED`,<br>`ABANDONED`              |
| Payment      | `status`        | `PENDING`, `SUCCESS`,<br>`FAILED`, `REFUNDED`           |
| Purchase     | `status`        | `ACTIVE`, `EXPIRED`,<br>`REFUNDED`                      |
| Notification | `type`          | `SYSTEM`, `PURCHASE`,<br>`COURSE_UPDATE`, `TEST_RESULT` |
| Inquiry      | `status`        | `OPEN`, `IN_PROGRESS`,<br>`RESOLVED`, `CLOSED`          |

---

## ## 9. Important Notes for Frontend

### ### 9.1 Cookie-Based Auth with CORS

When making API calls from the frontend, you **must** set ``withCredentials: true`` in your HTTP client (Axios, Fetch, etc.):

```
```javascript
// Axios example
axios.defaults.withCredentials = true;
axios.defaults.baseURL = 'http://localhost:3000/api/v1';

// Or per-request
axios.get('/users/me', { withCredentials: true });

// Fetch example
fetch('http://localhost:3000/api/v1/users/me', {
  credentials: 'include',
  headers: {
    'Authorization': `Bearer ${accessToken}`
  }
});
```
```

### ### 9.2 Token Refresh Flow

Implement an **Axios interceptor** (or equivalent) to handle token refresh:

1. If a request returns ``401``, call ``POST /auth/refresh-token``
2. If refresh succeeds, retry the original request with the new token
3. If refresh fails, redirect to login page

### ### 9.3 File Upload (Resource Upload)

Use ``FormData`` for the resource upload endpoint:

```
```javascript
const formData = new FormData();
```

```
formData.append('file', fileInput.files[0]);
formData.append('title', 'Study Notes Chapter 1');
formData.append('course', '665a...');
formData.append('resource_type', 'pdf');

axios.post('/resources', formData, {
  headers: { 'Content-Type': 'multipart/form-data' }
});
```
```

### ### 9.4 Razorpay Payment Integration

```
```javascript
// Step 1: Create order
const { data } = await axios.post('/payments/create-order', {
  item_id: courseId,
  item_type: 'Course',
  amount: 999
});

// Step 2: Open Razorpay checkout
const options = {
  key: 'YOUR_RAZORPAY_KEY_ID', // Get from backend team
  amount: data.data.amount,
  currency: data.data.currency,
  order_id: data.data.id,
  handler: async function (response) {
    // Step 3: Verify payment
    await axios.post('/payments/verify', {
      razorpay_order_id: response.razorpay_order_id,
      razorpay_payment_id: response.razorpay_payment_id,
      razorpay_signature: response.razorpay_signature
    });
    // Payment successful - user is now enrolled
  }
};

const rzp = new Razorpay(options);
rzp.open();
```
```



### ### 9.5 Test-Taking Flow

1. **\*\*Start Test:\*\*** `POST /attempts/start` → save the returned `attemptId`
2. **\*\*Save Answers:\*\*** `PUT /attempts/:attemptId/answer` (call on each answer)
3. **\*\*Submit Test:\*\*** `POST /attempts/:attemptId/submit` (manual submit)
4. **\*\*Get Results:\*\*** `POST /attempts/:attemptId/evaluate` (grades the test)
5. **\*\*View Results:\*\*** `GET /attempts/:attemptId` (fetch attempt details)

> Tests auto-submit after the duration expires. Build a countdown timer on the frontend.

### ### 9.6 Soft Deletes

Courses, MockTests, and Resources use **\*\*soft deletes\*\*** (`isDeleted: true`). They won't appear in list endpoints after deletion but are not permanently removed from the database.

### ### 9.7 ObjectId Format

All MongoDB ObjectId fields are **\*\*24-character hex strings\*\***. Example: `665a1234abcd5678ef901234`. Use this regex for frontend validation: `/^[0-9a-fA-F]{24}\$/`

### ### 9.8 Timestamps

All models include `createdAt` and `updatedAt` fields in **\*\*ISO 8601\*\*** format. Example: `2026-06-12T14:30:00.000Z`

---

### ## Complete Routes Summary Table

| Method       | Endpoint              | Access   |
|--------------|-----------------------|----------|
| Content-Type |                       |          |
| -----        | -----                 |          |
| -----        | -----                 |          |
| `GET`        | `/api/v1/healthcheck` | ● Public |
| JSON         |                       |          |

|               |                                    |        |
|---------------|------------------------------------|--------|
| POST          | /api/v1/auth/register              | Public |
| JSON          |                                    |        |
| POST          | /api/v1/auth/login                 | Public |
| JSON          |                                    |        |
| POST          | /api/v1/auth/logout                | Auth   |
| —             |                                    |        |
| POST          | /api/v1/auth/refresh-token         | Public |
| JSON / Cookie |                                    |        |
| POST          | /api/v1/auth/change-password       | Auth   |
| JSON          |                                    |        |
| POST          | /api/v1/auth/forgot-password       | Public |
| JSON          |                                    |        |
| POST          | /api/v1/auth/reset-password/:token | Public |
| JSON          |                                    |        |
| GET           | /api/v1/users/me                   | Auth   |
| —             |                                    |        |
| PATCH         | /api/v1/users/update-account       | Auth   |
| JSON          |                                    |        |
| PATCH         | /api/v1/users/avatar               | Auth   |
| JSON          |                                    |        |
| GET           | /api/v1/users                      | Admin  |
| —             |                                    |        |
| GET           | /api/v1/users/:id                  | Admin  |
| —             |                                    |        |
| PATCH         | /api/v1/users/:id/status           | Admin  |
| JSON          |                                    |        |
| DELETE        | /api/v1/users/:id                  | Admin  |
| —             |                                    |        |
| GET           | /api/v1/categories                 | Public |
| —             |                                    |        |
| GET           | /api/v1/categories/:id             | Public |
| —             |                                    |        |
| POST          | /api/v1/categories                 | Admin  |
| JSON          |                                    |        |
| PUT           | /api/v1/categories/:id             | Admin  |
| JSON          |                                    |        |
| DELETE        | /api/v1/categories/:id             | Admin  |
| —             |                                    |        |
| GET           | /api/v1/courses                    | Auth   |
| —             |                                    |        |

|                     |  |                                      |  |                                                                                             |
|---------------------|--|--------------------------------------|--|---------------------------------------------------------------------------------------------|
| `GET`               |  | `/api/v1/courses/:id`                |  |  Auth    |
| -                   |  |                                      |  |                                                                                             |
| `GET`               |  | `/api/v1/courses/my/enrolled`        |  |  Auth    |
| -                   |  |                                      |  |                                                                                             |
| `POST`              |  | `/api/v1/courses/:id/enroll`         |  |  Auth    |
| -                   |  |                                      |  |                                                                                             |
| `POST`              |  | `/api/v1/courses`                    |  |  Admin   |
| JSON                |  |                                      |  |                                                                                             |
| `PUT`               |  | `/api/v1/courses/:id`                |  |  Admin   |
| JSON                |  |                                      |  |                                                                                             |
| `DELETE`            |  | `/api/v1/courses/:id`                |  |  Admin   |
| -                   |  |                                      |  |                                                                                             |
| `PATCH`             |  | `/api/v1/courses/:id/publish`        |  |  Admin   |
| -                   |  |                                      |  |                                                                                             |
| `PATCH`             |  | `/api/v1/courses/:id/unpublish`      |  |  Admin   |
| -                   |  |                                      |  |                                                                                             |
| `GET`               |  | `/api/v1/resources/course/:courseId` |  |  Auth    |
| -                   |  |                                      |  |                                                                                             |
| `GET`               |  | `/api/v1/resources/:id/download`     |  |  Auth    |
| -                   |  |                                      |  |                                                                                             |
| `POST`              |  | `/api/v1/resources`                  |  |  Admin |
| multipart/form-data |  |                                      |  |                                                                                             |
| `DELETE`            |  | `/api/v1/resources/:id`              |  |  Admin |
| -                   |  |                                      |  |                                                                                             |
| `GET`               |  | `/api/v1/mock-tests`                 |  |  Auth  |
| -                   |  |                                      |  |                                                                                             |
| `GET`               |  | `/api/v1/mock-tests/:id`             |  |  Auth  |
| -                   |  |                                      |  |                                                                                             |
| `POST`              |  | `/api/v1/mock-tests`                 |  |  Admin |
| JSON                |  |                                      |  |                                                                                             |
| `PUT`               |  | `/api/v1/mock-tests/:id`             |  |  Admin |
| JSON                |  |                                      |  |                                                                                             |
| `DELETE`            |  | `/api/v1/mock-tests/:id`             |  |  Admin |
| -                   |  |                                      |  |                                                                                             |
| `PATCH`             |  | `/api/v1/mock-tests/:id/publish`     |  |  Admin |
| -                   |  |                                      |  |                                                                                             |
| `PATCH`             |  | `/api/v1/mock-tests/:id/unpublish`   |  |  Admin |
| -                   |  |                                      |  |                                                                                             |
| `POST`              |  | `/api/v1/mock-tests/:id/questions`   |  |  Admin |
| JSON                |  |                                      |  |                                                                                             |

|          |                                                |                                                                                             |
|----------|------------------------------------------------|---------------------------------------------------------------------------------------------|
| `POST`   | `/api/v1/mock-tests/:id/questions/bulk`        |  Admin   |
| JSON     |                                                |                                                                                             |
| `PUT`    | `/api/v1/mock-tests/:id/questions/:questionId` |  Admin   |
| JSON     |                                                |                                                                                             |
| `DELETE` | `/api/v1/mock-tests/:id/questions/:questionId` |  Admin   |
| -        |                                                |                                                                                             |
| `POST`   | `/api/v1/attempts/start`                       |  Auth    |
| JSON     |                                                |                                                                                             |
| `PUT`    | `/api/v1/attempts/:attemptId/answer`           |  Auth    |
| JSON     |                                                |                                                                                             |
| `POST`   | `/api/v1/attempts/:attemptId/submit`           |  Auth    |
| -        |                                                |                                                                                             |
| `POST`   | `/api/v1/attempts/:attemptId/evaluate`         |  Auth    |
| -        |                                                |                                                                                             |
| `GET`    | `/api/v1/attempts/:attemptId`                  |  Auth    |
| -        |                                                |                                                                                             |
| `POST`   | `/api/v1/payments/create-order`                |  Auth    |
| JSON     |                                                |                                                                                             |
| `POST`   | `/api/v1/payments/verify`                      |  Auth    |
| JSON     |                                                |                                                                                             |
| `GET`    | `/api/v1/notifications`                        |  Auth  |
| -        |                                                |                                                                                             |
| `PATCH`  | `/api/v1/notifications/:id/read`               |  Auth  |
| -        |                                                |                                                                                             |
| `GET`    | `/api/v1/dashboard/student`                    |  Auth  |
| -        |                                                |                                                                                             |
| `GET`    | `/api/v1/dashboard/admin`                      |  Admin |
| -        |                                                |                                                                                             |
| `GET`    | `/api/v1/leaderboard/:testId`                  |  Auth  |
| -        |                                                |                                                                                             |
| `POST`   | `/api/v1/inquiries`                            |  Auth  |
| JSON     |                                                |                                                                                             |
| `GET`    | `/api/v1/inquiries/my`                         |  Auth  |
| -        |                                                |                                                                                             |
| `GET`    | `/api/v1/inquiries`                            |  Admin |
| -        |                                                |                                                                                             |
| `PATCH`  | `/api/v1/inquiries/:id/reply`                  |  Admin |
| JSON     |                                                |                                                                                             |

---

**\*\*Total Routes: 47\*\***

*\*Document generated from source code analysis of mastermock\_backend.\**